ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng phần mềm quản lý khen thưởng tại Học viện Khoa học Quân sự

TRẦN ANH ĐỨC
duc.ta@sis.hust.edu.vn

Giảng viên hướng dẫn: ThS. Lê Đức Trung _____

Chữ kí GVHD

Chương trình đào tạo: Khoa học máy tính (IT1)

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 05/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng phần mềm quản lý khen thưởng tại Học viện Khoa học Quân sự

TRẦN ANH ĐỨC
duc.ta@sis.hust.edu.vn

Giảng viên hướng dẫn: ThS. Lê Đức Trung _____

Chữ kí GVHD

Chương trình đào tạo: Khoa học máy tính (IT1)

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 05/2026

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành tới các thầy cô giáo tại Đại học Bách Khoa Hà Nội và Trường Công nghệ Thông tin và Truyền thông – Đại học Bách Khoa Hà Nội, những người đã luôn tận tâm, giúp đỡ em trong suốt quá trình học tập. Đặc biệt, em xin gửi lời biết ơn sâu sắc đến thầy ThS. Lê Đức Trung đã tận tình hướng dẫn và hỗ trợ em trong suốt quá trình thực hiện ĐATN. Những lời khuyên và sự hỗ trợ của thầy là nguồn động viên lớn lao để em hoàn thành đồ án này.

Em cũng xin gửi lời cảm ơn tới tập thể cán bộ Học viện Khoa học Quân sự đã chia sẻ kiến thức nghiệp vụ về công tác thi đua – khen thưởng và phản hồi chi tiết về luồng nghiệp vụ trong quá trình xây dựng phần mềm.

Em xin cảm ơn gia đình, người thân, bạn bè – những người đã luôn đồng hành, yêu thương và tạo mọi điều kiện tốt nhất cho em trong suốt quá trình học tập tại Đại học Bách Khoa Hà Nội. Đặc biệt, em xin cảm ơn bố mẹ đã luôn ủng hộ, động viên em để em có thể hoàn thành mục tiêu học tập của mình.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Công tác thi đua, khen thưởng tại các đơn vị quân đội tuân thủ Luật Thi đua, Khen thưởng số 06/2022/QH15. Tại Học viện Khoa học Quân sự, riêng nhóm khen thưởng cá nhân hằng năm bao gồm chuỗi ba cấp Bằng khen của Bộ trưởng Bộ Quốc phòng (BKBTBQP), Chiến sĩ thi đua toàn quân (CSTĐTQ) và Bằng khen của Thủ tướng Chính phủ (BKTTCP) — mỗi cấp có chu kỳ trượt khác nhau (2, 3 và 7 năm) và yêu cầu đối chiếu nhiều cờ trong cửa sổ thời gian, làm cho việc xét chọn thủ công trên Excel dễ sai sót.

Đồ án xây dựng phần mềm web hỗ trợ Phòng Chính trị Học viện quản lý toàn bộ vòng đời của bảy nhóm khen thưởng (cá nhân hằng năm, đơn vị hằng năm, niên hạn, cống hiến, kỷ niệm chương, quân kỳ quyết thắng, nghiên cứu khoa học), bao quát từ tạo đề xuất, kiểm tra điều kiện chuỗi tự động, phê duyệt, lưu vết tới xuất quyết định. Phần mềm hướng tới rút ngắn thời gian xét điều kiện từ vài chục phút trên Excel xuống dưới một giây cho mỗi quân nhân, đồng thời ghi nhật ký đầy đủ phục vụ kiểm tra hậu kiểm.

Hệ thống được triển khai dưới dạng ứng dụng web vận hành trên mạng nội bộ (LAN) của Học viện, với phần frontend dùng Next.js 14 (App Router) và phần backend dùng Express + TypeScript trên cơ sở dữ liệu PostgreSQL truy cập qua Prisma ORM. Toàn bộ quy tắc xét chuỗi được trừu tượng hoá thành các đối tượng cấu hình `ChainAwardConfig` cho phép thêm tier mới mà không cần sửa mã nguồn xét điều kiện. Sản phẩm bao gồm 23 model dữ liệu, hơn 890 ca kiểm thử đơn vị (phủ rule chuỗi cá nhân và đơn vị) và 4 cấp phân quyền theo đặc thù quân sự (SuperAdmin, Admin, Manager, User).

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

Trần Anh Đức

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 1

1.1 Đặt vấn đề 1

1.2 Mục tiêu và phạm vi đề tài 2

1.3 Định hướng giải pháp 3

1.4 Bố cục đồ án 4

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU 6

2.1 Khảo sát hiện trạng 6

2.2 Tổng quan chức năng 7

2.2.1 Sơ đồ use case tổng quát 7

2.2.2 Phân rã: Quản lý quân nhân và lịch sử chức vụ 8

2.2.3 Phân rã: Quản lý đơn vị và chức vụ 8

2.2.4 Phân rã: Đề xuất khen thưởng theo bảy nhóm 9

2.2.5 Phân rã: Chuỗi danh hiệu hằng năm cá nhân 10

2.2.6 Phân rã: Chuỗi danh hiệu hằng năm đơn vị 11

2.2.7 Phân rã: Nhập và xuất dữ liệu Excel 11

2.2.8 Phân rã: Quản trị hệ thống 12

2.2.9 Quy trình nghiệp vụ — ba luồng quan trọng 12

2.3 Đặc tả use case chi tiết 13

2.4 Yêu cầu phi chức năng 18

2.4.1 Hiệu năng và trải nghiệm người dùng 18

2.4.2 Bảo mật 19

2.4.3 Tính nhất quán và chính xác của dữ liệu 20

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG 22

3.1 Next.js 14 — Khung phát triển frontend 22

3.2 TypeScript và Express — Nền tảng backend 22

3.3 PostgreSQL và Prisma ORM 23

3.4 Ant Design, Tailwind CSS và shadcn/ui — Bộ giao diện ba lớp 24

3.5 Socket.IO — Kênh thông báo thời gian thực 25

3.6 JSON Web Token với cơ chế làm mới luân phiên 25

3.7 Joi và Zod — Kiểm tra dữ liệu ở hai phía 26

3.8 Jest — Khung kiểm thử đơn vị 26

3.9 ExcelJS — Công cụ nhập và xuất Excel 27

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG 28

4.1 Thiết kế kiến trúc 28

4.1.1 Lựa chọn kiến trúc phần mềm 28

4.1.2 Tổng quan hệ thống 29

4.1.3 Thiết kế tổng quan các gói 29

4.1.4 Thiết kế chi tiết gói nghiệp vụ Khen thưởng cá nhân hằng năm 30

4.1.5 Thiết kế lớp 31

4.1.6 Biểu đồ tuần tự 32

4.1.7 Thiết kế cơ sở dữ liệu 34

4.2 Thiết kế chi tiết 38

4.2.1 Thiết kế giao diện 38

4.2.2 Hệ thống thiết kế (Design System) 38

4.3 Xây dựng ứng dụng 39

4.3.1 Thư viện và công cụ sử dụng 39

4.3.2 Kết quả đạt được 39

4.3.3 Minh hoạ các chức năng chính 40

4.3.4 Tóm tắt các thành phần kỹ thuật trọng yếu 41

4.4 Kiểm thử 42
4.4.1 Kiểm thử đơn vị bằng Jest 42
4.4.2 Kiểm thử hộp đen các chức năng nghiệp vụ 43
4.4.3 Kiểm thử tương thích trình duyệt 45
4.5 Triển khai 46
4.5.1 Yêu cầu phần cứng và phần mềm 46
4.5.2 Hướng dẫn triển khai môi trường phát triển 46
4.5.3 Triển khai sản xuất với PM2 và Nginx 47
4.5.4 Sao lưu và giám sát 47
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT 48
5.1 Tự động hoá kiểm tra điều kiện chuỗi danh hiệu 48
5.2 Quy trình đề xuất – phê duyệt số hoá kèm nhật ký kiểm toán 49
5.3 Tính lại tổng thể và gợi ý đề nghị chủ động 50
5.4 Nhập liệu Excel hàng loạt có giao dịch và tiền kiểm 51
5.5 Phân quyền theo cây đơn vị, kiểm toán đầy đủ và sao lưu định kỳ 51
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 53
6.1 Kết luận 53
6.2 Hướng phát triển 54
TÀI LIỆU THAM KHẢO 58
PHỤ LỤC 60
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP 60
A.1 Ngành học 61
A.2 Đánh dấu (bullet) và đánh số (numering) 61
A.3 Cách thêm bảng 62
A.4 Chèn hình ảnh 62
A.5 Tài liệu tham khảo 63

A.6 Cách viết phương trình và công thức toán học..... 63

A.7 Qui cách đóng quyển..... 63

B. ĐẶC TẢ USE CASE BỔ SUNG..... 65

B.1 Đặc tả use case “Quản lý đơn vị (CQĐV / ĐVTT)”..... 65

B.2 Đặc tả use case “Sao lưu dữ liệu”..... 66

B.3 Đặc tả use case “Quản lý tài khoản”..... 67

B.4 Đặc tả use case “Xem nhật ký hệ thống” 68

DANH MỤC HÌNH VẼ

Hình 2.1 Sơ đồ use case tổng quát hệ thống PM QLKT 8
Hình 2.2 Sơ đồ use case phân rã chức năng Quản lý quân nhân 8
Hình 2.3 Sơ đồ use case phân rã chức năng Quản lý đơn vị và chức vụ . 9
Hình 2.4 Sơ đồ use case phân rã chức năng Đề xuất khen thưởng 10
Hình 2.5 Sơ đồ use case phân rã chức năng Chuỗi danh hiệu hằng năm cá nhân 10
Hình 2.6 Sơ đồ use case phân rã chức năng Chuỗi danh hiệu hằng năm đơn vị 11
Hình 2.7 Sơ đồ use case phân rã chức năng Nhập – xuất Excel 12
Hình 2.8 Sơ đồ use case phân rã chức năng Quản trị hệ thống 12

Hình 4.1 Sơ đồ kiến trúc phân tầng sáu lớp 29
Hình 4.2 Sơ đồ kiến trúc tổng quan FE — BE — DB 29
Hình 4.3 Sơ đồ gói phía frontend 30
Hình 4.4 Sơ đồ gói phía backend 30
Hình 4.5 Sơ đồ gói chi tiết module khen thưởng cá nhân hằng năm . . . 31
Hình 4.6 Sơ đồ lớp năm module nghiệp vụ trọng yếu 32
Hình 4.7 Bảy sơ đồ tuần tự cho các luồng nghiệp vụ chính 34
Hình 4.8 Sơ đồ ERD tổng thể của 23 model 34
Hình 4.9 Màn hình đăng nhập 41
Hình 4.10 Trang chủ Admin với bốn thẻ thống kê và biểu đồ phân bố . . 41

Hình A.1 Internet vạn vật 62
Hình A.2 Qui cách đóng quyển đồ án 64
Hình A.3 Qui cách đóng quyển đồ án 64

DANH MỤC BẢNG BIỂU

Bảng 2.1 Đặc tả use case UC-01 Đăng nhập 14
Bảng 2.2 Đặc tả use case UC-02 Tạo mới quân nhân 14
Bảng 2.3 Đặc tả use case UC-03 Tạo đề xuất khen thưởng 15
Bảng 2.4 Đặc tả use case UC-04 Phê duyệt đề xuất 16
Bảng 2.5 Đặc tả use case UC-05 Kiểm tra điều kiện chuỗi và sinh gợi ý 17
Bảng 2.6 Đặc tả use case UC-06 Nhập tệp Excel hàng loạt 18

Bảng 3.1 So sánh ba ORM phổ biến trên Node.js 24

Bảng 4.1 Schema bảng `QuanNhan` (Personnel) 35
Bảng 4.2 Schema bảng `BangDeXuat` (Proposal) 35
Bảng 4.3 Schema bảng `DanhHieuHangNam` (Annual Award) 36
Bảng 4.4 Schema bảng `LichSuChucVu` (Position History) 36
Bảng 4.5 Schema bảng `SystemLog` (Audit Log) 37
Bảng 4.6 Schema bảng `HoSoHangNam` (Annual Profile — Derived) . 37
Bảng 4.7 Thư viện và công cụ chính 39
Bảng 4.8 Tổng kết kiểm thử đơn vị 43
Bảng 4.9 Kiểm thử chức năng Đăng nhập 43
Bảng 4.10 Kiểm thử chức năng Quản lý quân nhân 43
Bảng 4.11 Kiểm thử chức năng Đề xuất khen thưởng 44
Bảng 4.12 Kiểm thử chức năng Phê duyệt đề xuất 44
Bảng 4.13 Kiểm thử chức năng Tính lại điều kiện chuỗi 44
Bảng 4.14 Kiểm thử chức năng Nhập Excel hàng loạt 45
Bảng 4.15 Kiểm thử chức năng Quản lý đơn vị và chức vụ 45
Bảng 4.16 Kiểm thử chức năng Quản trị (sao lưu, nhật ký) 45
Bảng 4.17 Kiểm thử tương thích trình duyệt 46

Bảng A.1 Table to test captions and labels. 62

Bảng B.1 Đặc tả use case UC-07 Quản lý đơn vị 65
Bảng B.2 Đặc tả use case UC-08 Sao lưu dữ liệu 66
Bảng B.3 Đặc tả use case UC-09 Quản lý tài khoản 67
Bảng B.4 Đặc tả use case UC-10 Xem nhật ký hệ thống 68

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng
BKBTBQP	Bằng khen của Bộ trưởng Bộ Quốc phòng
BKTTCP	Bằng khen của Thủ tướng Chính phủ
BQP	Bộ Quốc phòng
CQĐV	Cơ quan đơn vị (cấp cha)
CSDL	Cơ sở dữ liệu
CSTĐCS	Chiến sĩ thi đua cơ sở
CSTĐTQ	Chiến sĩ thi đua toàn quân
ERD	Sơ đồ thực thể – quan hệ
HCBVTQ	Huân chương Bảo vệ Tổ quốc
HCCSVV	Huy chương Chiến sĩ vẻ vang
HCQKQT	Huy chương Quân kỳ quyết thắng
HTML	Ngôn ngữ đánh dấu siêu văn bản
HVKHQS	Học viện Khoa học Quân sự
JWT	Mã thông báo web JSON
KNC	Kỷ niệm chương Vì sự nghiệp xây dựng QĐNDVN
LAN	Mạng nội bộ
NCKH	Nghiên cứu khoa học
ORM	Ánh xạ đối tượng – quan hệ
QLKT	Quản lý khen thưởng
QĐNDVN	Quân đội Nhân dân Việt Nam
RBAC	Kiểm soát truy cập theo vai trò
SV	Sinh viên
ĐATN	Đồ án tốt nghiệp
ĐVQT	Đơn vị quyết thắng
ĐVTT	Đơn vị trực thuộc

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này trình bày bối cảnh, sự cần thiết và mục tiêu của đồ án Xây dựng phần mềm Quản lý Khen thưởng tại Học viện Khoa học Quân sự (PM QLKT). Sau khi nêu vấn đề thực tiễn ở mục 1.1, mục 1.2 xác định mục tiêu cùng phạm vi đề tài, mục 1.3 trình bày định hướng giải pháp tổng quát; cuối chương, mục 1.4 mô tả bố cục các chương còn lại của báo cáo.

1.1 Đặt vấn đề

Công tác thi đua – khen thưởng tại các đơn vị quân đội được điều chỉnh bởi Luật Thi đua, Khen thưởng số 06/2022/QH15 [1] cùng các nghị định và thông tư hướng dẫn. Trong khi nhiều cơ quan đã chuyển sang phần mềm điện tử, công tác xét khen thưởng tại Học viện Khoa học Quân sự (HVKHQS) hiện vẫn dựa chủ yếu trên các tệp Microsoft Excel chia sẻ qua mạng nội bộ kèm bản giấy ký xác nhận. Phương thức này phát sinh ba nhóm khó khăn cụ thể.

Thứ nhất, hệ thống danh hiệu có yếu tố **chuỗi** với cửa sổ thời gian trượt phức tạp. Để xét Bằng khen của Bộ trưởng Bộ Quốc phòng (BKBTBQP), một quân nhân phải duy trì danh hiệu Chiến sĩ thi đua cơ sở (CSTĐCS) liên tục đủ hai năm. Xét Chiến sĩ thi đua toàn quân (CSTĐTQ) cần ba năm CSTĐCS liên tục, đồng thời có ít nhất một BKBTBQP trong cửa sổ ba năm cuối. Xét Bằng khen của Thủ tướng Chính phủ (BKTTCP) cần bảy năm CSTĐCS liên tục, đúng ba BKBTBQP và đúng hai CSTĐTQ trong cửa sổ bảy năm cuối, kèm yêu cầu mỗi năm có thành tích nghiên cứu khoa học. Khi rà soát thủ công, cán bộ phải tra chéo nhiều bảng tính để xác định một danh hiệu cũ còn nằm trong cửa sổ trượt hay đã rơi khỏi cửa sổ — vừa chậm vừa dễ sai sót.

Thứ hai, hệ thống khen thưởng tại Học viện gồm **bảy nhóm khen thưởng** với nghiệp vụ tách bạch: khen thưởng cá nhân hằng năm, khen thưởng đơn vị hằng năm, niên hạn (Huy chương Chiến sĩ vẻ vang theo mốc 10/15/20 năm phục vụ), cống hiến (Huân chương Bảo vệ Tổ quốc dựa trên thời gian giữ chức vụ theo từng nhóm hệ số), kỷ niệm chương (vì sự nghiệp xây dựng Quân đội nhân dân Việt Nam, một lần duy nhất theo giới tính), Huy chương Quân kỳ quyết thắng (25 năm phục vụ) và thành tích nghiên cứu khoa học. Mỗi nhóm có quy tắc đầu vào, mẫu quyết định và quy trình duyệt riêng, vì vậy duy trì bằng tệp tính khó đảm bảo nhất quán giữa các nhóm.

Thứ ba, **dấu vết kiểm toán** của các thao tác trên Excel không được lưu lại. Khi một bản ghi bị chỉnh sửa hoặc xoá, không có cơ chế truy được người thực hiện,

mốc thời gian và lý do — gây khó khăn cho công tác hậu kiểm và đối chiếu khi cấp trên yêu cầu. Việc phân quyền truy cập tệp dùng chung cũng chỉ ở mức thư mục, không phản ánh được phân cấp thực tế của Học viện (cấp Phòng, cấp Khoa/Trung tâm, cá nhân quân nhân).

Trước những hạn chế trên, việc xây dựng một phần mềm chuyên dụng cho công tác quản lý khen thưởng tại HVKHQS là cần thiết. Phần mềm phải đảm nhiệm được toàn bộ vòng đời nghiệp vụ, từ nhập dữ liệu lịch sử qua Excel, tự động kiểm tra điều kiện chuỗi, xử lý đề xuất – phê duyệt cho đến xuất quyết định và lưu vết, đồng thời tuân thủ phân quyền nhiều cấp đặc thù môi trường quân sự.

Đóng góp chính của đồ án tập trung vào bốn khía cạnh kỹ thuật: (i) thiết kế kiến trúc phân tầng có lớp Repository tách khỏi Service; (ii) thuật toán xét chuỗi danh hiệu (BKBTBQP, CSTĐTQ, BKTTCP cá nhân; BKBTBQP, BKTTCP đơn vị) được trừu tượng hoá qua đối tượng cấu hình `ChainAwardConfig`; (iii) cơ chế đề xuất – phê duyệt theo Strategy pattern cho bảy nhóm khen thưởng kèm nhật ký kiểm toán đầy đủ; (iv) bộ kiểm thử đơn vị gồm 890 ca phủ toàn bộ rule chuỗi và các kịch bản phê duyệt. Bốn khía cạnh này được trình bày chi tiết từ Chương 3 đến Chương 5.

1.2 Mục tiêu và phạm vi đề tài

Mục tiêu chính của đồ án là xây dựng một hệ thống web hỗ trợ Phòng Chính trị Học viện Khoa học Quân sự thực hiện đầy đủ các nhiệm vụ sau:

1. Quản lý danh sách quân nhân, đơn vị (theo cấu trúc cây Cơ quan đơn vị – Đơn vị trực thuộc), chức vụ và lịch sử biến động chức vụ.
2. Cho phép cán bộ Phòng Chính trị tạo các đợt đề xuất khen thưởng cho cả bảy nhóm danh hiệu, hỗ trợ chọn nhanh nhiều quân nhân thông qua lọc theo đơn vị, năm, giới tính, mốc thời gian phục vụ.
3. Tự động kiểm tra điều kiện chuỗi (BKBTBQP, CSTĐTQ, BKTTCP đối với cá nhân; BKBTBQP và BKTTCP đối với đơn vị) và sinh thông điệp gợi ý dễ hiểu cho cán bộ.
4. Hỗ trợ quy trình phê duyệt nhiều bước, lưu file quyết định PDF kèm số quyết định, đồng thời ghi nhật ký kiểm toán toàn bộ thao tác mutate.
5. Cung cấp công cụ nhập dữ liệu lịch sử qua Excel có tiền kiểm và giao dịch (transaction) đảm bảo nguyên tử tính, cùng công cụ xuất danh sách khen thưởng theo nhiều tiêu chí.
6. Phân quyền theo bốn cấp tương ứng đặc thù tổ chức: SuperAdmin (bộ phận kỹ thuật và công nghệ thông tin, quản trị hạ tầng và an ninh hệ thống), Admin

(Phòng Chính trị Học viện, toàn quyền nghiệp vụ), Manager (cán bộ chỉ huy đơn vị, đề xuất và quản lý quân nhân nội bộ), User (cán bộ – học viên – sĩ quan, tra cứu hồ sơ cá nhân).

7. Hỗ trợ sao lưu cơ sở dữ liệu định kỳ và quản lý nhật ký hệ thống.

Phạm vi đề tài giới hạn ở các nhiệm vụ liên quan trực tiếp đến công tác xét khen thưởng. Phần mềm vận hành tập trung trên máy chủ của Học viện và được truy cập qua mạng nội bộ (LAN), không xuất bản ra Internet công cộng nhằm đáp ứng yêu cầu bảo mật đặc thù môi trường quân sự. Đồ án **không bao gồm** các nghiệp vụ sau: chế độ tiền lương và phụ cấp, hồ sơ y tế, đánh giá cán bộ định kỳ, các quy trình hành chính khác. Việc tích hợp với hệ thống quản lý nhân sự tổng thể của Bộ Quốc phòng (nếu có) cũng không nằm trong phạm vi, do hai khía cạnh đó vượt ngoài mục tiêu trọng tâm về khen thưởng.

1.3 Định hướng giải pháp

Phần mềm được xây dựng theo mô hình client – server với hai thành phần độc lập triển khai trên cùng một máy chủ vật lý hoặc tách máy.

- **Frontend** dùng Next.js 14 với mô hình App Router [2], viết bằng TypeScript, sử dụng Ant Design làm thư viện thành phần chính, kết hợp Tailwind CSS cho bố cục linh hoạt và shadcn/ui cho các thành phần đặc biệt.
- **Backend** dùng Express [3] trên Node.js, viết bằng TypeScript, tổ chức theo kiến trúc phân tầng (Route → Middleware → Controller → Service → Repository → Prisma). Lớp Repository được tách riêng giúp giảm sự phụ thuộc trực tiếp vào Prisma Client trong tầng Service, tạo điều kiện thay thế ORM trong tương lai mà không phải viết lại logic nghiệp vụ.
- **Cơ sở dữ liệu** chọn PostgreSQL [4] truy cập qua Prisma ORM [5]. Toàn bộ schema (23 model) đã được mô hình hoá trong tệp `schema.prisma` duy nhất với migration tự động.
- **Xác thực và phân quyền** dùng cặp JSON Web Token [6] (Access Token thời hạn ngắn 30 phút + Refresh Token 7 ngày có rotation) lưu trữ phía client tại `localStorage` và phía máy chủ tại cột `TaiKhoan.refreshToken` để có khả năng thu hồi. Bốn vai trò được áp đặt qua middleware `requireRole` chạy trước mỗi endpoint nhạy cảm.
- **Thông báo thời gian thực** sử dụng Socket.IO [7], chủ yếu thông báo kết quả nhập Excel lớn, biến cố phê duyệt và xoá đề xuất cho người tạo đề xuất, đồng thời báo cho Phòng Chính trị khi có đề xuất mới.
- **Kiểm thử** dùng Jest + ts-jest [8]. Tổng cộng có 75 bộ kiểm thử đơn vị với 890

ca kiểm thử bao phủ rule chuỗi (cá nhân và đơn vị), kịch bản phê duyệt và xếp hạng huy chương.

Quy tắc chuỗi danh hiệu được trừu tượng hoá thành đối tượng cấu hình `ChainAwardConfig` (thuộc tính: mã danh hiệu, số năm chu kỳ, số cờ tiền điều kiện cần thiết, có yêu cầu nghiên cứu khoa học hay không, tính chất trọn đời hay lặp lại, tên cột cờ trong cơ sở dữ liệu, nhãn chuỗi). Hai hằng số `PERSONAL_CHAIN_AWARDS` và `UNIT_CHAIN_AWARDS` chứa danh sách cấu hình tương ứng cho cá nhân và đơn vị. Một hàm xét điều kiện duy nhất `checkChainEligibility` đọc các đối tượng này để kết luận, qua đó việc thêm tier mới chỉ cần bổ sung phần tử vào mảng cấu hình mà không phải sửa logic.

1.4 Bố cục đồ án

Phần còn lại của báo cáo được tổ chức thành năm chương theo mô hình quen thuộc cho đồ án tốt nghiệp ngành Khoa học máy tính tại Trường Công nghệ Thông tin và Truyền thông – Đại học Bách Khoa Hà Nội.

Chương 2 trình bày kết quả khảo sát thực tế công tác khen thưởng tại Học viện Khoa học Quân sự, xác định các tác nhân tham gia hệ thống, mô tả tổng quan chức năng qua sơ đồ use case và các sơ đồ hoạt động cho những quy trình nghiệp vụ chính (đề xuất – phê duyệt, kiểm tra điều kiện chuỗi, nhập Excel hàng loạt). Phần đặc tả chi tiết sáu use case trọng yếu được trình bày dưới dạng bảng. Cuối cùng, chương đưa ra các yêu cầu phi chức năng về hiệu năng, bảo mật và tính nhất quán.

Chương 3 giới thiệu các thành phần công nghệ được lựa chọn cho dự án và lý do chọn từng thành phần trong ngữ cảnh PM QLKT, bao gồm Next.js 14 App Router, Express + TypeScript, PostgreSQL + Prisma, các thư viện giao diện (Ant Design, Tailwind, shadcn/ui), Socket.IO, JWT, Joi, Zod, Jest và ExcelJS.

Chương 4 đi sâu vào kiến trúc phân tầng của hệ thống, sơ đồ gói cho frontend và backend, sơ đồ lớp cho năm module nghiệp vụ trọng yếu, các sơ đồ tuần tự cho luồng đăng nhập, tạo đề xuất, phê duyệt và tính lại điều kiện chuỗi, thiết kế cơ sở dữ liệu (ERD đầy đủ và mô tả schema), phần xây dựng thực tế kèm các đoạn mã minh hoạ, kết quả kiểm thử và hướng dẫn triển khai bằng PM2 và Nginx.

Chương 5 lần lượt phân tích năm điểm khác biệt mà sản phẩm mang lại so với phương thức thủ công hiện tại. Mỗi điểm trình bày theo cấu trúc thực trạng – giải pháp – kết quả đo lường định lượng được rút ra từ kiểm thử và dữ liệu mẫu.

Chương 6 tổng kết các nhiệm vụ đã hoàn thành, các bài học rút ra sau quá trình thực hiện, đồng thời đề xuất bảy hướng phát triển cho phiên bản kế tiếp, bao gồm hỗ trợ các danh hiệu cao hơn BKTTCP, tích hợp ký số quyết định, xây dựng ứng

dụng di động cho người dùng cấp USER và mở rộng khả năng phân tích dữ liệu cho cấp Bộ.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này trình bày kết quả khảo sát hiện trạng công tác thi đua – khen thưởng tại Học viện Khoa học Quân sự (HVKHQS), từ đó phân tích các yêu cầu chức năng và phi chức năng của phần mềm. Mục 2.1 mô tả thực trạng và các nhóm khó khăn nổi bật. Mục 2.2 đưa ra cấu trúc tác nhân và sơ đồ use case tổng quát cùng tám sơ đồ phân rã. Mục 2.3 đặc tả chi tiết sáu use case trọng yếu nhất. Mục 2.4 tổng hợp các yêu cầu phi chức năng về hiệu năng, bảo mật và tính nhất quán dữ liệu.

2.1 Khảo sát hiện trạng

Trước khi triển khai phần mềm chuyên dụng, công tác thi đua – khen thưởng tại Học viện Khoa học Quân sự được tổ chức xoay quanh ba nguồn dữ liệu chính: hồ sơ giấy lưu tại Phòng Chính trị, các tệp Microsoft Excel chia sẻ qua thư mục mạng nội bộ và sổ theo dõi tay của từng đơn vị. Trong các cuộc trao đổi với cán bộ phụ trách thi đua, nhóm thực hiện đồ án ghi nhận một số đặc điểm đáng chú ý.

Vai trò của các bên tham gia. Cấp đề xuất đầu tiên thường là cán bộ chỉ huy đơn vị hoặc cán bộ chính trị cấp cơ quan, nắm danh sách quân nhân thuộc phạm vi quản lý và lập danh sách đề nghị theo từng năm hoặc theo các đợt đặc biệt. Cấp xét duyệt là Phòng Chính trị Học viện, kiểm tra điều kiện và lập tờ trình gửi Ban Giám đốc. Cấp ký quyết định là Giám đốc Học viện đối với các danh hiệu thuộc thẩm quyền và là cấp trên (Tổng cục II, Bộ Quốc phòng, Thủ tướng Chính phủ) đối với các danh hiệu vượt thẩm quyền. Quân nhân – đối tượng được khen thưởng – thường tiếp nhận thông tin một cách thụ động qua thông báo bằng văn bản hoặc bảng tin đơn vị.

Khối lượng dữ liệu. Khi tổng hợp ngược lại quá khứ, một quân nhân thường có nhiều bản ghi danh hiệu thuộc các nhóm khác nhau: từ chuỗi danh hiệu hằng năm tích luỹ trong toàn bộ thời gian phục vụ, đến các huy chương niên hạn (10/15/20 năm), kỷ niệm chương, quân kỳ quyết thắng và các khen thưởng đột xuất. Tổng khối lượng bản ghi cho cán bộ trên 25 năm phục vụ lên tới 30–40 bản ghi mỗi quân nhân, nhân với quy mô vài nghìn cán bộ – học viên – sĩ quan, dữ liệu lịch sử cần lưu trữ vào khoảng vài trăm nghìn bản ghi.

Nhóm khó khăn nổi bật. Bốn vấn đề được phản ánh nhiều nhất qua khảo sát là:

1. Tra cứu lịch sử khen thưởng cá nhân tốn thời gian do phải mở nhiều tệp Excel.
2. Đối chiếu chuỗi danh hiệu hằng năm theo cửa sổ trượt khó vì rule có nhiều ngoại lệ (chu kỳ lặp lại, BKTTCP cá nhân là một lần duy nhất, BKBTBQP

của chu kỳ trước rơi ra khỏi cửa sổ 3 năm khi xét CSTĐTQ chu kỳ mới).

3. Thiếu cơ chế kiểm tra trùng đề xuất khi nhiều cán bộ cùng phụ trách một đơn vị.
4. Nhật ký kiểm toán hầu như không có, dẫn tới khó truy hồi khi phát hiện sai sót.

Từ kết quả khảo sát, các đặc thù nghiệp vụ trên đòi hỏi một hệ thống có khả năng (a) lưu trữ tập trung và liên kết được các loại bản ghi, (b) tự động kiểm tra điều kiện theo các quy tắc đã được mã hoá, (c) phân quyền theo cây tổ chức của Học viện và (d) sinh nhật ký kiểm toán cho mọi thao tác có khả năng làm thay đổi dữ liệu nghiệp vụ.

2.2 Tổng quan chức năng

Phần này trình bày cấu trúc tác nhân và các nhóm chức năng chính của hệ thống thông qua sơ đồ use case tổng quát và các sơ đồ phân rã chi tiết.

2.2.1 Sơ đồ use case tổng quát

Hệ thống có bốn nhóm tác nhân tương ứng với bốn cấp phân quyền nội bộ:

- **SuperAdmin** đại diện cho bộ phận kỹ thuật và công nghệ thông tin. Nhóm này không tham gia trực tiếp vào nghiệp vụ xét khen thưởng mà tập trung vào quản trị hạ tầng, sao lưu – khôi phục dữ liệu, theo dõi nhật ký an ninh và quản trị các tài khoản cấp quản trị.
- **Admin** là cán bộ thuộc Phòng Chính trị Học viện. Nhóm này có toàn quyền nghiệp vụ: tạo và phê duyệt đề xuất, quản lý quân nhân – đơn vị – chức vụ, gắn số quyết định, tải lên tệp PDF quyết định và xuất các báo cáo.
- **Manager** là cán bộ chỉ huy hoặc cán bộ chính trị cấp đơn vị, phụ trách lập đề xuất khen thưởng cho phạm vi đơn vị mình quản lý và theo dõi trạng thái phê duyệt. Manager có thể thao tác trên dữ liệu quân nhân thuộc đơn vị nhưng không thể chuyển quân nhân giữa các đơn vị (chức năng này chỉ thuộc Admin).
- **User** là cán bộ – học viên – sĩ quan đăng nhập để tra cứu hồ sơ khen thưởng cá nhân, xem các chu kỳ chuỗi đang đến mốc đề nghị và nhận thông báo về kết quả phê duyệt.

Hệ thống cung cấp 15 nhóm chức năng chính: Đăng nhập, Quản lý tài khoản, Quản lý quân nhân, Quản lý đơn vị, Khen thưởng hằng năm, Khen thưởng niên hạn, Khen thưởng cống hiến, Khen thưởng thành tích khoa học, Khen thưởng đột xuất, Đề xuất khen thưởng, Kiểm tra điều kiện và gợi ý, Thông báo thời gian thực, Xem nhật ký hệ thống, Sao lưu và khôi phục, Báo cáo và thống kê. Mối quan hệ

giữa từng tác nhân và các nhóm chức năng được thể hiện ở Hình 2.1.

Hình 2.1: Sơ đồ use case tổng quát hệ thống PM QLKT

2.2.2 Phân rã: Quản lý quân nhân và lịch sử chức vụ

Quản lý quân nhân là một trong những module tương tác cao nhất của hệ thống. Quân nhân là thực thể trung tâm liên kết tới gần như mọi bảng nghiệp vụ khác (danh hiệu hằng năm, niên hạn, cống hiến, thành tích khoa học, đột xuất, đề xuất). Phân rã chức năng gồm các use case con:

- **Tạo mới quân nhân** — chỉ Admin có quyền. Yêu cầu nhập tối thiểu họ tên, số CCCD, ngày sinh, giới tính, ngày nhập ngũ, đơn vị (CQĐV hoặc ĐVTT), chức vụ. Hệ thống kiểm tra trùng CCCD trước khi ghi vào cơ sở dữ liệu.
- **Cập nhật thông tin** — Admin sửa được toàn bộ; Manager chỉ sửa được các trường ngoại trừ đơn vị và CCCD; bản thân quân nhân (User) chỉ sửa được địa chỉ, số điện thoại, email.
- **Chuyển đơn vị** — chỉ Admin. Khi chuyển, hệ thống cập nhật số lượng quân nhân của đơn vị cũ (giảm 1) và đơn vị mới (tăng 1) trong cùng một transaction. Quan trọng là toàn bộ lịch sử khen thưởng được giữ nguyên — không tạo bản ghi mới hay xoá bản ghi cũ.
- **Quản lý lịch sử chức vụ** — Admin nhập danh sách các giai đoạn quân nhân giữ chức vụ với hệ số tương ứng. Dữ liệu này là đầu vào cho thuật toán xét khen thưởng cống hiến (HCBVTQ).
- **Nhập / xuất Excel** — cho phép tải lên hoặc tải xuống danh sách quân nhân theo định dạng quy chuẩn. Thao tác nhập đi qua hai bước (xem trước, xác nhận) sẽ được mô tả chi tiết tại 2.2.7.

Hình 2.2: Sơ đồ use case phân rã chức năng Quản lý quân nhân

2.2.3 Phân rã: Quản lý đơn vị và chức vụ

Cấu trúc tổ chức của Học viện được mô hình hoá thành cây hai cấp. Bảng `CoQuanDonVi` (CQĐV) lưu các đơn vị cấp cha; bảng `DonViTrucThuoc` (ĐVTT) lưu các đơn vị con thuộc một CQĐV duy nhất. Khi nhập thông tin quân nhân, một trong hai trường `co_quan_don_vi_id` hoặc `don_vi_truc_thuoc_id` sẽ được điền — không bao giờ điền cả hai cùng lúc.

Module quản lý đơn vị bao gồm các use case con: tạo mới CQĐV, tạo mới ĐVTT

thuộc một CQĐV cụ thể, cập nhật thông tin, xoá (chỉ thực hiện được khi không còn quân nhân nào thuộc đơn vị) và xem cây tổ chức dưới dạng tree-view trên giao diện. Bảng `ChucVu` quản lý danh mục chức vụ kèm hệ số `he_so_chuc_vu` (giá trị từ 0.5 đến 1.2 tuỳ chức danh) và liên kết tới CQĐV hoặc ĐVTT mà chức vụ đó thuộc về. Hệ số chức vụ là dữ liệu đầu vào trực tiếp cho thuật toán tính tháng cống hiến theo nhóm hệ số 0.7 / 0.8 / 0.9–1.0.

Hình 2.3: Sơ đồ use case phân rã chức năng Quản lý đơn vị và chức vụ

2.2.4 Phân rã: Đề xuất khen thưởng theo bảy nhóm

Đề xuất khen thưởng là điểm khởi đầu của mọi quy trình ghi nhận thành tích. Hệ thống hỗ trợ bảy loại đề xuất riêng biệt, mỗi loại có dữ liệu đầu vào và quy tắc xét điều kiện khác nhau:

1. **Cá nhân hằng năm** (`CA_NHAN_HANG_NAM`): danh hiệu CSTT, CSTĐCS, BKBTBQP, CSTĐTQ, BKTTCP cộng các cờ tương ứng và số quyết định cấp cao hơn.
2. **Đơn vị hằng năm** (`DON_VI_HANG_NAM`): danh hiệu ĐVQT, ĐVTT cộng các cờ BKBTBQP / BKTTCP cấp đơn vị.
3. **Niên hạn** (`NIEN_HAN`): Huy chương Chiến sĩ vẻ vang theo các mốc 10 / 15 / 20 năm phục vụ tích luỹ tới năm xét.
4. **Cống hiến** (`CONG_HIEN`): Huân chương Bảo vệ Tổ quốc dựa trên thời gian giữ chức vụ thuộc các nhóm hệ số khác nhau.
5. **Kỷ niệm chương** (`KNC_VSNXD_QDNDVN`): Kỷ niệm chương vì sự nghiệp xây dựng QĐNDVN — một lần duy nhất theo giới tính (nam 25 năm phục vụ, nữ 20 năm).
6. **Quân kỳ quyết thắng** (`HC_QKQT`): Huy chương Quân kỳ quyết thắng cho cán bộ phục vụ đủ 25 năm.
7. **Thành tích khoa học** (`NCKH`): Khen thưởng đề tài khoa học hoặc sáng kiến kinh nghiệm.

Tất cả bảy loại đều dùng chung một tầng dữ liệu là bảng `BangDeXuat` với các trường JSON `data_danh_hieu`, `data_thanh_tich`, `data_nien_han`, `data_cong_hien` để lưu chi tiết theo từng loại. Việc dùng JSON ở đây có chủ đích: cho phép schema linh hoạt theo từng loại đề xuất mà không phải tạo nhiều bảng riêng cho mỗi loại. Mỗi loại đề xuất được gắn với một đối tượng cài đặt giao diện `ProposalStrategy` chịu trách nhiệm xây dựng, kiểm tra và nhập dữ liệu;

chi tiết được trình bày tại Chương 4.

Quy trình tạo đề xuất gồm ba bước trên giao diện: chọn loại đề xuất và năm/tháng → chọn quân nhân hoặc đơn vị → thiết lập danh hiệu cụ thể cho từng đối tượng. Bước hai có thể chọn nhiều quân nhân cùng lúc thông qua bộ lọc theo đơn vị, năm sinh, giới tính, mốc thời gian phục vụ.

Hình 2.4: Sơ đồ use case phân rã chức năng Đề xuất khen thưởng

2.2.5 Phân rã: Chuỗi danh hiệu hằng năm cá nhân

Đây là module nghiệp vụ phức tạp nhất của hệ thống do có quy tắc chuỗi với cửa sổ trượt và có một danh hiệu lifetime. Cấu hình chuỗi cá nhân gồm ba tier (BKBTBQP, CSTĐTQ, BKTTCP) tương ứng ba phần tử trong mảng `PERSONAL_CHAIN_AWARDS`:

- **BKBTBQP** có chu kỳ 2 năm (yêu cầu 2 năm CSTĐCS liên tục), không yêu cầu cờ tiền điều kiện, có yêu cầu thành tích NCKH mỗi năm, không phải lifetime (có thể nhận lại sau mỗi chu kỳ 2 năm).
- **CSTĐTQ** có chu kỳ 3 năm, yêu cầu đúng 1 BKBTBQP nằm trong cửa sổ trượt 3 năm gần nhất tính từ năm trước năm xét, yêu cầu NCKH mỗi năm, không phải lifetime.
- **BKTTCP** có chu kỳ 7 năm, yêu cầu đúng 3 BKBTBQP và đúng 2 CSTĐTQ trong cửa sổ trượt 7 năm cuối, yêu cầu NCKH mỗi năm, **là lifetime** đối với cá nhân (mỗi quân nhân chỉ nhận một lần trong toàn bộ thời gian phục vụ).

Use case chính của module này là **Kiểm tra điều kiện và gợi ý**: hệ thống tự tính toán dựa trên dữ liệu lịch sử và sinh ra hai loại thông tin cho mỗi quân nhân: (a) danh sách các tier đủ điều kiện đề nghị trong năm hiện tại, kèm thông điệp giải thích; (b) số liệu cửa sổ trượt (số BKBTBQP đã có trong cửa sổ 3 năm hoặc 7 năm, số CSTĐTQ đã có trong cửa sổ 7 năm, độ dài chuỗi CSTĐCS hiện tại). Cán bộ Phòng Chính trị có thể tham khảo gợi ý này để chốt danh sách đề xuất.

Use case phụ là **Tính lại tổng thể**: khi một bản ghi danh hiệu hằng năm cũ được chỉnh sửa hoặc xoá, hệ thống có thể chạy lại thuật toán cho toàn bộ quân nhân hoặc một quân nhân riêng lẻ, đảm bảo dữ liệu suy diễn (`HoSoHangNam`) luôn nhất quán với dữ liệu nguồn (`DanhHieuHangNam`).

Hình 2.5: Sơ đồ use case phân rã chức năng Chuỗi danh hiệu hằng năm cá nhân

2.2.6 Phân rã: Chuỗi danh hiệu hằng năm đơn vị

Chuỗi đơn vị có cấu trúc đơn giản hơn cá nhân do không có tier CSTĐTQ và không yêu cầu thành tích NCKH. Cấu hình `UNIT_CHAIN_AWARDS` chỉ gồm hai tier:

- **BKBTBQP đơn vị** chu kỳ 2 năm, yêu cầu 2 năm ĐVQT liên tục, không có cờ tiền điều kiện, không phải lifetime.
- **BKTTCP đơn vị** chu kỳ 7 năm, yêu cầu ít nhất 3 BKBTBQP đơn vị trong cửa sổ trượt 7 năm cuối, không phải lifetime (đơn vị có thể nhận lại sau mỗi chu kỳ 7 năm).

Khác biệt quan trọng với chuỗi cá nhân là điều kiện cờ "ít nhất 3 BKBTBQP" thay vì "đúng 3 BKBTBQP". Lý do là một đơn vị có thể nhận lại BKTTCP sau mỗi chu kỳ 7 năm, nên việc đếm cờ trong cửa sổ trượt cần linh hoạt hơn so với danh hiệu lifetime của cá nhân.

Hình 2.6: Sơ đồ use case phân rã chức năng Chuỗi danh hiệu hằng năm đơn vị

2.2.7 Phân rã: Nhập và xuất dữ liệu Excel

Học viện có lượng dữ liệu lịch sử lớn được lưu trên Excel trước khi triển khai phần mềm; do đó chức năng nhập Excel hàng loạt là cầu nối quan trọng giúp hệ thống tiếp nhận dữ liệu cũ mà không phải nhập tay từng bản ghi. Quy trình gồm bốn use case con:

- **Tải tệp mẫu** — hệ thống sinh tệp Excel có cấu trúc cột chuẩn theo từng loại nghiệp vụ (quân nhân, danh hiệu hằng năm, lịch sử chức vụ, niên hạn, ...) cùng các Data Validation rule cho phép chọn từ danh sách thả xuống ngay trong tệp.
- **Xem trước** — sau khi cán bộ điền dữ liệu và tải tệp lên, hệ thống đọc nội dung từng dòng, đối chiếu với schema Joi và các quy tắc nghiệp vụ, sau đó hiển thị kết quả gồm danh sách dòng hợp lệ và danh sách dòng có lỗi (kèm chỉ số dòng và mô tả lỗi). Bước này không ghi vào cơ sở dữ liệu.
- **Xác nhận** — cán bộ kiểm tra danh sách hợp lệ rồi xác nhận. Hệ thống mở một transaction Prisma, ghi tuần tự từng dòng; nếu xảy ra lỗi tại bất kỳ dòng nào, toàn bộ thao tác được hủy bỏ và cơ sở dữ liệu giữ nguyên trạng thái trước khi nhập.
- **Xuất danh sách** — cho phép xuất kết quả khen thưởng theo nhiều tiêu chí (theo năm, theo đơn vị, theo loại danh hiệu) ra tệp Excel có định dạng phù hợp với mẫu báo cáo nội bộ của Học viện.

Hình 2.7: Sơ đồ use case phân rã chức năng Nhập – xuất Excel

2.2.8 Phân rã: Quản trị hệ thống

Module quản trị bao quát các chức năng nền tảng phục vụ vận hành hệ thống. Bao gồm:

- **Quản lý tài khoản** — SuperAdmin tạo tài khoản Admin; Admin tạo tài khoản Manager và User; mỗi tài khoản gắn với một quân nhân duy nhất qua khoá ngoại `quan_nhan_id`.
- **Đặt lại mật khẩu** — Admin có thể đặt lại mật khẩu cho Manager / User về giá trị mặc định.
- **Nhật ký hệ thống** (`SystemLog`) — ghi lại mọi thao tác thay đổi dữ liệu với metadata gồm thời điểm, người thực hiện, loại tài nguyên, mã định danh tài nguyên, mô tả thao tác và payload trước – sau khi thay đổi.
- **Sao lưu và khôi phục** — SuperAdmin cấu hình lịch sao lưu định kỳ qua khu vực DevZone; hệ thống dùng `pg_dump` sinh tệp SQL và lưu tại thư mục `backups/`. Thao tác khôi phục cũng được thực hiện qua giao diện này.
- **Cấu hình hệ thống** (`SystemSetting`) — bật/tắt các cờ tính năng, ví dụ `allow_notify_import` cho phép gửi thông báo khi nhập Excel hàng loạt.

Tất cả các thao tác trong module quản trị đều có lưu nhật ký riêng để đảm bảo trách nhiệm cá nhân và truy hồi.

Hình 2.8: Sơ đồ use case phân rã chức năng Quản trị hệ thống

2.2.9 Quy trình nghiệp vụ — ba luồng quan trọng

Bên cạnh sơ đồ use case, ba quy trình nghiệp vụ phức tạp nhất được mô hình hoá thêm bằng sơ đồ hoạt động (activity diagram) để làm rõ luồng xử lý từng bước.

Quy trình A — Đề xuất → Phê duyệt một đợt khen thưởng. Manager chọn loại đề xuất và năm xét, hệ thống lọc danh sách quân nhân/đơn vị thuộc phạm vi quản lý. Manager chọn các đối tượng đủ điều kiện, thiết lập danh hiệu cụ thể và bấm gửi đề xuất. Hệ thống lưu vào bảng `BangDeXuat` ở trạng thái `PENDING`, đồng thời gửi thông báo realtime tới các Admin. Admin mở danh sách đề xuất chờ duyệt, kiểm tra lại từng đối tượng, có thể chỉnh sửa danh hiệu trước khi phê duyệt. Khi phê duyệt, hệ thống mở transaction để (a) đổi trạng thái đề xuất sang `APPROVED`, (b) ghi các bản ghi danh hiệu/khen thưởng vào bảng tương ứng

(như `DanhHieuHangNam`, `KhenThuongHCCSVV`, ...), (c) gắn số quyết định và đường dẫn tệp PDF, (d) ghi nhật ký kiểm toán, (e) gửi thông báo tới Manager đã đề xuất và (f) tính lại các hồ sơ suy diễn liên quan.

Quy trình B — Tính điều kiện chuỗi cho một quân nhân. Khi cán bộ truy cập trang hồ sơ của một quân nhân, hệ thống nạp toàn bộ bản ghi `DanhHieuHangNam` của quân nhân đó, sắp xếp giảm dần theo năm và chạy hàm `computeChainContext`. Hàm này tính ra ba chỉ số chính: (a) độ dài chuỗi CSTĐCS hiện tại (`streakCstdcs`), (b) năm gần nhất có cờ BKBTBQP / CSTĐTQ / BKTTCP, (c) số cờ trong cửa sổ trượt tương ứng (3 năm cho CSTĐTQ, 7 năm cho BKTTCP). Sau đó với mỗi tier trong `PERSONAL_CHAIN_AWARDS`, hàm `checkChainEligibility` đối chiếu số liệu thực tế với cấu hình tier và trả về kết quả `eligible: boolean` kèm thông điệp `reason` bằng tiếng Việt.

Quy trình C — Nhập tệp Excel hàng loạt. Cán bộ Admin tải lên tệp Excel chứa danh sách bản ghi cần nhập. Hệ thống đọc tệp ở backend, tách thành các sheet (vd: `QuanNhan`, `DanhHieuHangNam`, `ThanhTichKhoaHoc`) và đọc từng dòng. Mỗi dòng được kiểm tra qua schema Joi và các quy tắc nghiệp vụ liên quan (vd: CCCD không được trùng, đơn vị phải tồn tại). Các dòng hợp lệ và lỗi được tổng hợp vào kết quả `ImportPreview` trả về frontend. Cán bộ xem trước, đối chiếu, có thể quay lại sửa tệp Excel rồi tải lên lại. Khi xác nhận, backend mở transaction Prisma, ghi tuần tự các dòng hợp lệ; nếu có bất kỳ lỗi nào tại thời điểm ghi, transaction được rollback hoàn toàn.

2.3 Đặc tả use case chi tiết

Phần này đặc tả sáu use case trọng yếu của hệ thống dưới dạng bảng theo template UML chuẩn. Cấu trúc mỗi bảng gồm sáu phần: định danh và tên, các tác nhân tham gia, tiền điều kiện cần thoả mãn trước khi thực hiện, luồng sự kiện chính theo từng bước, các luồng thay thế thường gặp và hậu điều kiện sau khi thực hiện thành công. Sáu use case được lựa chọn vì phản ánh đúng các nghiệp vụ phức tạp nhất và có nhiều tương tác giữa các module. Đặc tả đầy đủ các use case còn lại được trình bày tại Phụ lục B.

Bảng 2.1: Đặc tả use case UC-01 Đăng nhập

Mục	**Nội dung**
Tên	Đăng nhập hệ thống
ID	UC-01
Tác nhân	SuperAdmin, Admin, Manager, User
Tiền điều kiện	Người dùng đã có tài khoản hợp lệ trong bảng `TaiKhoan`; trình duyệt cho phép `localStorage`.
Luồng chính	(1) Người dùng truy cập trang `/login`. (2) Nhập tên đăng nhập và mật khẩu. (3) Bấm nút "Đăng nhập". (4) Hệ thống tra cứu tài khoản theo `username`, đối chiếu mật khẩu băm bằng bcrypt. (5) Hệ thống phát hành Access Token (30 phút) và Refresh Token (7 ngày), trả về cho frontend ghi vào `localStorage`. (6) Chuyển hướng tới trang chủ tương ứng vai trò.
Luồng thay thế	(4a) Tài khoản không tồn tại hoặc mật khẩu sai → hiển thị thông báo "Tài khoản hoặc mật khẩu không đúng" mà không tiết lộ vế nào sai. (4b) Tài khoản bị khoá → hiển thị "Tài khoản đã bị tạm khoá, liên hệ quản trị viên". (5a) Sau ba lần sai liên tiếp, áp giới hạn `rateLimiter` 5 phút cho địa chỉ IP.
Hậu điều kiện	Phiên đăng nhập được thiết lập; mọi yêu cầu HTTP tiếp theo gửi kèm token được middleware `verifyToken` xác thực; nhật ký đăng nhập được ghi vào `SystemLog` với `action = LOGIN`.

Bảng 2.2: Đặc tả use case UC-02 Tạo mới quân nhân

Mục	**Nội dung**
Tên	Tạo mới hồ sơ quân nhân
ID	UC-02
Tác nhân	Admin
Tiền điều kiện	Đã đăng nhập với vai trò Admin; cây tổ chức (CQĐV/ĐVTT) đã có dữ liệu.
Luồng chính	(1) Admin truy cập "Quản lý quân nhân" → bấm "Thêm mới". (2) Điền các trường bắt buộc (họ tên, CCCD, ngày sinh, giới tính, ngày nhập ngũ, đơn vị, chức vụ). (3) Bấm "Lưu". (4) Frontend xác thực schema Zod; backend xác thực schema Joi. (5) Backend kiểm tra trùng CCCD trong bảng `QuanNhan`. (6) Ghi bản ghi mới và cập nhật bộ đếm `so_luong` của đơn vị. (7) Trả về kết quả thành công, frontend cập nhật bảng danh sách.
Luồng thay thế	(5a) CCCD đã tồn tại → backend trả về lỗi 409 với thông điệp "Số CCCD đã tồn tại trong hệ thống". (4a) Một trường bắt buộc bị thiếu → frontend chặn ngay tại form trước khi gọi API. (6a) Đơn vị được chọn không tồn tại (do bị xoá song song) → backend trả về lỗi 404 và rollback transaction.
Hậu điều kiện	Bản ghi mới có mặt trong `QuanNhan`; số lượng quân nhân của đơn vị tăng 1; nhật ký `action = CREATE, resource = personnel` được ghi với payload chứa CCCD và họ tên.

Bảng 2.3: Đặc tả use case UC-03 Tạo đề xuất khen thưởng

Mục	**Nội dung**
Tên	Tạo đề xuất khen thưởng cho một đợt
ID	UC-03
Tác nhân	Admin, Manager
Tiền điều kiện	Đã đăng nhập; có ít nhất một quân nhân thuộc phạm vi quản lý của Manager (nếu là Manager).
Luồng chính	(1) Truy cập "Đề xuất khen thưởng" → "Tạo đề xuất mới". (2) Bước 1: chọn loại đề xuất (1 trong 7 loại) và nhập năm/tháng xét. (3) Bước 2: hệ thống lọc và hiển thị danh sách quân nhân hoặc đơn vị thuộc phạm vi quản lý phù hợp với loại đề xuất; người dùng tích chọn các đối tượng. (4) Bước 3: thiết lập danh hiệu cụ thể cho từng đối tượng (vd: chọn HCCSVV hạng Ba/Nhì/Nhất); hệ thống có thể tự gợi ý dựa trên dữ liệu lịch sử. (5) Tải lên các tệp đính kèm nếu có. (6) Bấm "Gửi đề xuất". (7) Backend xác thực dữ liệu, tạo bản ghi `BangDeXuat` ở trạng thái `PENDING` cùng các trường JSON tương ứng. (8) Phát thông báo realtime tới các Admin liên quan.
Luồng thay thế	(4a) Có quân nhân không đủ điều kiện trong danh sách → hệ thống hiển thị cảnh báo nhưng vẫn cho phép gửi nếu Manager chấp nhận trách nhiệm. (3a) Manager chọn quân nhân ngoài phạm vi đơn vị → backend trả lỗi 403. (7a) Có lỗi mạng hoặc backend không phản hồi → frontend lưu nháp ở localStorage để người dùng không mất dữ liệu.
Hậu điều kiện	Bản ghi `BangDeXuat` được tạo với trạng thái `PENDING`; thông báo realtime tới Admin; `SystemLog` ghi `action = CREATE, resource = proposals` cùng tóm tắt loại và số lượng đối tượng.

Bảng 2.4: Đặc tả use case UC-04 Phê duyệt đề xuất

Mục	**Nội dung**
Tên	Phê duyệt một đề xuất khen thưởng
ID	UC-04
Tác nhân	Admin
Tiền điều kiện	Đề xuất đang ở trạng thái `PENDING`; Admin có quyền phê duyệt loại đề xuất tương ứng.
Luồng chính	(1) Admin mở chi tiết đề xuất từ danh sách "Chờ duyệt". (2) Xem lại danh sách đối tượng và danh hiệu được đề xuất. (3) Có thể chỉnh sửa danh hiệu của một số đối tượng (vd: hạ từ Hạng Nhất xuống Hạng Nhì). (4) Nhập số quyết định và tải lên tệp PDF quyết định. (5) Bấm "Phê duyệt". (6) Backend mở transaction Prisma: chuyển trạng thái đề xuất sang `APPROVED`; ghi các bản ghi danh hiệu/khen thưởng vào bảng tương ứng; gắn số quyết định; tính lại hồ sơ suy diễn cho các quân nhân bị ảnh hưởng. (7) Phát thông báo realtime tới Manager đã đề xuất và các quân nhân được khen thưởng. (8) Ghi nhật ký với payload trước–sau khi thay đổi.
Luồng thay thế	(5a) Đề xuất bị thay đổi bởi Admin khác cùng lúc → hệ thống dùng lock optimistic, trả về lỗi 409 và yêu cầu tải lại trước khi phê duyệt. (4a) Số quyết định trùng với một quyết định đã có trong `FileQuyetDinh` → backend cảnh báo nhưng cho phép tiếp tục nếu Admin xác nhận. (6a) Có lỗi tại bước ghi danh hiệu (vd: vi phạm rule lifetime BKTTCP) → toàn bộ transaction rollback, đề xuất giữ nguyên trạng thái `PENDING`. (5b) Admin từ chối thay vì phê duyệt → đổi trạng thái sang `REJECTED` kèm lý do, gửi thông báo tới Manager.
Hậu điều kiện	Trạng thái đề xuất là `APPROVED` hoặc `REJECTED`; nếu `APPROVED`, các bản ghi khen thưởng tương ứng đã có mặt trong cơ sở dữ liệu; hồ sơ suy diễn được tính lại; thông báo đã gửi; nhật ký đầy đủ.

Bảng 2.5: Đặc tả use case UC-05 Kiểm tra điều kiện chuỗi và sinh gợi ý

Mục	**Nội dung**
Tên	Tính lại hồ sơ hằng năm và sinh gợi ý đề nghị
ID	UC-05
Tác nhân	Admin (kích hoạt chủ động); System (tự động sau mỗi lần phê duyệt).
Tiền điều kiện	Quân nhân tồn tại; có ít nhất một bản ghi trong `DanhHieuHangNam`.
Luồng chính	(1) Hệ thống truy vấn toàn bộ `DanhHieuHangNam` của quân nhân, sắp xếp giảm dần theo `nam`. (2) Gọi hàm `computeChainContext(rows, currentYear)` để dẫn xuất ngữ cảnh chuỗi. (3) Với mỗi tier trong `PERSONAL_CHAIN_AWARDS`, gọi `checkChainEligibility(tier, context, hasReceivedLifetime)`. Hàm trả về `{ eligible, reason }` bằng tiếng Việt. (4) Lưu kết quả vào bảng suy diễn `HoSoHangNam` (cập nhật hoặc tạo mới); ghi đè các trường `du_dieu_kien_bkbqp`, `du_dieu_kien_cstdtq`, `du_dieu_kien_bkttcp`, `goi_y`. (5) Trả về kết quả cho frontend hiển thị.
Luồng thay thế	(3a) Quân nhân đã nhận BKTTCP → hàm trả về `eligible: false` kèm lý do "Đã có BKTTCP. Phần mềm chưa hỗ trợ các danh hiệu cao hơn BKTTCP". (3b) Chuỗi CSTĐCS chưa đủ chu kỳ → trả về thông điệp gợi ý số năm cần tích luỹ thêm. (3c) Năm hiện tại đúng vào mốc lỡ đợt → trả về thông điệp giải thích cách thức xét lại ở chu kỳ kế tiếp.
Hậu điều kiện	`HoSoHangNam` của quân nhân có dữ liệu nhất quán với `DanhHieuHangNam`; trang hồ sơ cá nhân hiển thị các tier đủ điều kiện và thông điệp gợi ý chi tiết.

Bảng 2.6: Đặc tả use case UC-06 Nhập tệp Excel hàng loạt

Mục	**Nội dung**
Tên	Nhập danh sách quân nhân hoặc danh hiệu hằng năm từ Excel
ID	UC-06
Tác nhân	Admin
Tiền điều kiện	Đã có tệp mẫu Excel tải xuống từ hệ thống và điền dữ liệu theo cấu trúc cột chuẩn.
Luồng chính	(1) Admin truy cập "Nhập Excel" → chọn loại nghiệp vụ. (2) Tải lên tệp `.xlsx` qua form. (3) Backend đọc tệp bằng ExcelJS, kiểm tra tên các sheet, đọc tuần tự từng dòng. (4) Mỗi dòng được áp schema Joi và các quy tắc nghiệp vụ. (5) Tổng hợp kết quả thành đối tượng `{ valid: [...], errors: [...] }`. (6) Trả về frontend hiển thị bảng xem trước với chỉ số dòng, lỗi cụ thể, các dòng hợp lệ tô xanh. (7) Admin kiểm tra, nếu đồng ý thì bấm "Xác nhận nhập". (8) Backend mở transaction Prisma, ghi tuần tự các dòng `valid`. (9) Nếu mọi thao tác thành công, commit transaction và phát sự kiện realtime tổng kết.
Luồng thay thế	(3a) Tệp không đúng định dạng `.xlsx` hoặc thiếu sheet bắt buộc → trả về lỗi 400 với mô tả cụ thể. (4a) Có dòng lỗi trong giai đoạn xem trước → admin có thể tải tệp xuống, sửa, tải lại — không tiêu tốn thời gian transaction. (8a) Có lỗi xảy ra trong giai đoạn ghi → toàn bộ transaction rollback.
Hậu điều kiện	Các bản ghi trong tệp được ghi vào cơ sở dữ liệu; nhật ký `action = IMPORT` được ghi với payload tổng kết số dòng thành công và số dòng lỗi; thông báo realtime gửi tới Admin khởi tạo.

2.4 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng đã trình bày ở các mục trước, hệ thống cần đáp ứng một số yêu cầu phi chức năng để đảm bảo vận hành ổn định, an toàn và phù hợp với đặc thù môi trường quân sự. Các yêu cầu này được nhóm lại thành ba khía cạnh: hiệu năng và trải nghiệm người dùng, bảo mật, tính nhất quán dữ liệu.

2.4.1 Hiệu năng và trải nghiệm người dùng

Hệ thống được thiết kế để vận hành ổn định trong môi trường mạng nội bộ Học viện với khoảng vài chục người dùng đồng thời ở thời điểm cao điểm (đợt xét khen thưởng cuối năm). Các tiêu chí định lượng được đặt ra như sau.

Thời gian phản hồi của các trang chính (trang chủ, danh sách quân nhân, danh sách đề xuất) cần dưới hai giây ở điều kiện kết nối LAN bình thường. Các thao tác phức tạp như tính lại hồ sơ chuỗi cho một quân nhân cần dưới một giây; tính lại tổng thể cho 3000 quân nhân được phép kéo dài đến vài phút nhưng phải có thanh tiến trình hiển thị qua kênh Socket.IO.

Bảng dữ liệu lớn (vd: danh sách toàn bộ quân nhân, danh sách khen thưởng theo năm) được phân trang phía máy chủ với mặc định 20 dòng/trang và giới hạn tối đa 100 dòng/trang. Frontend áp dụng kỹ thuật cập nhật lạc quan (optimistic UI): các thao tác cập nhật trạng thái thường gặp (đánh dấu đã đọc thông báo, tích chọn quân nhân vào đề xuất) thay đổi ngay trên giao diện trước khi backend trả về xác nhận, qua đó giảm cảm giác chờ đợi cho người dùng.

Khu vực dữ liệu thường truy vấn nhưng ít thay đổi (cây CQĐV/ĐVTT, danh mục chức vụ, danh mục danh hiệu) được nạp một lần vào React Context và chia sẻ trong toàn bộ ứng dụng — tránh việc mỗi component tự gọi API riêng dẫn tới yêu cầu trùng lặp.

2.4.2 Bảo mật

Bảo mật là yêu cầu được ưu tiên cao nhất do hệ thống lưu trữ dữ liệu liên quan đến cán bộ và sĩ quan trong môi trường quân sự. Bảy nhóm biện pháp được áp dụng song song.

Thứ nhất — Xác thực bằng JWT có Refresh rotation. Mỗi phiên đăng nhập tạo ra cặp Access Token (30 phút) và Refresh Token (7 ngày). Frontend lưu cả hai token vào `localStorage` và đính kèm Access Token vào header `Authorization: Bearer` ở mỗi yêu cầu. Backend còn lưu Refresh Token vào cột `TaiKhoan.refreshToken` của cơ sở dữ liệu để có khả năng thu hồi từ phía máy chủ — middleware `verifyToken` kiểm tra cả chữ ký JWT và sự tồn tại của Refresh Token trong cơ sở dữ liệu, từ đó cho phép buộc đăng xuất bằng cách xoá cột này. Khi Access Token hết hạn, frontend gọi điểm cuối `/auth/refresh`; backend phát hành đồng thời cặp Access + Refresh Token mới và ghi đè Refresh Token cũ trong cơ sở dữ liệu (rotation), khiến mọi lần dùng Refresh Token cũ sau đó đều thất bại. Cơ chế này hạn chế nguy cơ một Refresh Token bị đánh cắp vẫn dùng được lâu dài.

Thứ hai — Băm mật khẩu bằng bcrypt với hệ số chi phí 10. Mật khẩu trước khi lưu vào bảng `TaiKhoan` không bao giờ được lưu ở dạng văn bản thô. Hệ số 10 được chọn để cân bằng giữa thời gian xác thực (khoảng 80 ms trên máy chủ phát triển) và độ khó tấn công vét cạn nếu cơ sở dữ liệu bị rò rỉ [9].

Thứ ba — Phân quyền bốn cấp qua middleware `requireRole`. Mỗi route nhạy cảm đi qua middleware kiểm tra vai trò của người dùng có nằm trong danh sách cho phép hay không. Phân quyền không chỉ dừng ở vai trò mà còn ở phạm vi quản lý: Manager chỉ truy vấn được dữ liệu của các quân nhân thuộc đơn vị mình quản lý, kiểm soát qua middleware `unitFilter`.

Thứ tư — Xác thực dữ liệu hai phía. Frontend dùng Zod [10] để xác thực ngay tại form, ngăn người dùng gửi đi dữ liệu sai định dạng. Backend dùng Joi [11] để xác thực lại tại điểm cuối; kể cả khi frontend bị bỏ qua (vd: tấn công gửi yêu cầu thẳng), backend vẫn từ chối dữ liệu không hợp lệ. Tuỳ chọn `stripUnknown: true` của Joi loại bỏ các trường ngoài schema để tránh người dùng đẩy lén các thuộc tính không được phép.

Thứ năm — Bảo vệ tầng vận chuyển và tầng ứng dụng. Toàn bộ kết nối tới server qua HTTPS (TLS 1.2 trở lên) khi triển khai sản xuất. Header bảo mật được áp đặt qua middleware Helmet (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options). CORS cấu hình chặt chẽ chỉ cho phép tên miền của frontend chính. Rate limiter chia hai cấp: `authLimiter` giới hạn 30 yêu cầu / IP / 5 phút cho các điểm cuối xác thực (đăng nhập, đặt lại mật khẩu) với chế độ `skipSuccessfulRequests` để không chặn nhầm người dùng hợp lệ; `writeLimiter` giới hạn 30 yêu cầu / IP / 15 phút cho các thao tác ghi nhạy cảm khác.

Thứ sáu — Nhật ký kiểm toán đầy đủ. Mọi thao tác có thể làm thay đổi dữ liệu nghiệp vụ (CREATE, UPDATE, DELETE, IMPORT, APPROVE, REJECT, LOGIN, LOGOUT, RESET_PASSWORD) đều ghi vào bảng `SystemLog` với metadata gồm thời điểm, mã người thực hiện, vai trò, loại tài nguyên (`resource`), mã định danh tài nguyên (`resource_id`) và mô tả tiếng Việt do hàm builder sinh tự động. Nhật ký liên quan đến `resource = backup` chỉ SuperAdmin xem được; các vai trò thấp hơn không thấy ngay cả tiêu đề.

Thứ bảy — Sao lưu định kỳ và khôi phục. Hệ thống tự động chạy `pg_dump` theo lịch cấu hình (mặc định mỗi 24 giờ vào 02:00 sáng) và lưu tệp SQL vào thư mục `backups/`. Mỗi tệp có tên theo mẫu `YYYY-MM-DD_HH-mm-ss.sql`. SuperAdmin có thể tải xuống, xoá hoặc khôi phục từ một tệp sao lưu cụ thể qua khu vực DevZone.

2.4.3 Tính nhất quán và chính xác của dữ liệu

Trong nghiệp vụ khen thưởng, một sai lệch nhỏ giữa các bản ghi liên quan có thể dẫn tới quyết định sai (vd: bỏ sót quân nhân đủ điều kiện hoặc đề xuất nhầm chu kỳ). Vì vậy, tính nhất quán dữ liệu được đặt ngang hàng với bảo mật.

Giao dịch ACID cho mọi thao tác phức tạp. Các thao tác đụng nhiều bảng được gói trong transaction Prisma. Phê duyệt một đề xuất chứa 50 quân nhân thực hiện 50 lần ghi `DanhHieuHangNam` cộng với cập nhật trạng thái đề xuất, ghi nhật ký, gắn quyết định — toàn bộ trong một transaction để bảo đảm: hoặc tất cả thành công, hoặc cơ sở dữ liệu trở về trạng thái ban đầu nếu một phép ghi gặp lỗi.

Kiểm tra điều kiện chuỗi ở hai tầng. Hàm `checkChainEligibility` được gọi cả trong hàm tính lại hồ sơ (`computeEligibilityFlags`) và trong hàm kiểm tra phê duyệt (`checkAwardEligibility`). Hai tầng phải khớp nhau: nếu có sự khác biệt, một phía sẽ chấp nhận đề xuất mà phía kia hiển thị "không đủ điều kiện" — gây mất niềm tin của cán bộ vào hệ thống. Để bảo đảm điều này, cả hai hàm được kiểm thử bằng Jest với cùng một bộ kịch bản.

Tính lại hồ sơ suy diễn sau mỗi thay đổi nguồn. Các bảng `HoSoHangNam`, `HoSoNienHan`, `HoSoCongHien` lưu các kết quả tính toán (cờ đủ điều kiện, gợi ý). Bất kỳ thao tác nào tạo, sửa hoặc xoá bản ghi nguồn đều kích hoạt hàm tính lại tương ứng. Việc này xảy ra tự động bên trong cùng transaction để đảm bảo dữ liệu suy diễn không bao giờ "đi sau" dữ liệu nguồn.

Tránh tham chiếu sai khoá ngoại. Mọi quan hệ giữa các bảng đều được khai báo qua `@relation` của Prisma kèm chính sách `onDelete: Restrict` cho các quan hệ trọng yếu (vd: không cho xoá `QuanNhan` khi vẫn còn bản ghi `DanhHieuHangNam`). Đối với một số bảng phụ trợ ít quan trọng (vd: thông báo), chính sách `onDelete: Cascade` được áp để tự dọn sạch khi xoá đối tượng cha.

Kiểm tra tính duy nhất qua chỉ mục unique. Các trường có yêu cầu duy nhất (`cccd` của quân nhân, `username` của tài khoản, tổ hợp `quan_nhan_id + nam` cho `DanhHieuHangNam`) đều được khai báo `@unique` ở schema. Khi vi phạm, Prisma trả về lỗi P2002 mà backend bắt và biến thành thông điệp tiếng Việt thân thiện (vd: "Quân nhân này đã có bản ghi danh hiệu cho năm 2024, vui lòng chỉnh sửa thay vì tạo mới").

Kết chương. Chương này đã trình bày kết quả khảo sát hiện trạng và phân tích yêu cầu cho phần mềm PM QLKT. Bốn nhóm tác nhân với các quyền hạn khác biệt cùng 15 nhóm chức năng đã được mô hình hoá qua sơ đồ use case tổng quát và tám sơ đồ phân rã chi tiết. Sáu use case trọng yếu được đặc tả đầy đủ phục vụ giai đoạn thiết kế. Phần yêu cầu phi chức năng đặt ra ba khía cạnh: hiệu năng và trải nghiệm người dùng, bảo mật bảy lớp và tính nhất quán dữ liệu — tất cả là kim chỉ nam cho các quyết định công nghệ và kiến trúc trong các chương tiếp theo.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Chương này giới thiệu các thành phần công nghệ chính được lựa chọn cho phần mềm Quản lý Khen thưởng (PM QLKT). Mỗi mục được trình bày theo bố cục thống nhất gồm ba phần: tổng quan ngắn gọn về công nghệ, đặc trưng kỹ thuật quan trọng và lý do lựa chọn trong ngữ cảnh cụ thể của đồ án. Cách trình bày này giúp người đọc thấy rõ động cơ của từng quyết định kỹ thuật thay vì chỉ liệt kê đặc điểm tổng quát.

3.1 Next.js 14 — Khung phát triển frontend

Next.js là một khung phát triển ứng dụng web do hãng Vercel phát triển trên nền React. Phiên bản 14 ra mắt tháng 11 năm 2023 đánh dấu việc App Router chính thức trở thành mô hình định tuyến mặc định, thay thế cho cấu trúc Pages Router ở các phiên bản trước [2]. App Router cho phép tổ chức ứng dụng dưới dạng cây thư mục bên trong thư mục `app/`, mỗi thư mục con tương ứng với một đoạn URL và có thể chứa các tệp `page.tsx`, `layout.tsx`, `loading.tsx`, `error.tsx` để biểu diễn trạng thái tương ứng.

Đặc trưng kỹ thuật đáng chú ý nhất ở App Router là khái niệm Server Components — các thành phần React mặc định chạy phía máy chủ, có thể truy vấn cơ sở dữ liệu hoặc gọi API nội bộ trực tiếp trong hàm render mà không cần state hay effect. Các thành phần cần xử lý sự kiện, hooks hoặc thư viện DOM mới khai báo `'use client'` ở đầu tệp. Cách tách biệt rõ ràng này giảm khối lượng JavaScript gửi xuống trình duyệt và đơn giản hoá việc lấy dữ liệu trên trang chỉ đọc.

Trong PM QLKT, App Router phù hợp với đặc thù phân vùng giao diện theo bốn vai trò người dùng. Các trang dành cho mỗi vai trò được đặt tại `app/super-admin/`, `app/admin/`, `app/manager/`, `app/user/`; mỗi nhánh có thể có `layout.tsx` riêng để áp các kiểm tra phân quyền tổng thể. Một ví dụ cụ thể là trang Bảng điều khiển dành cho ADMIN tại `app/admin/dashboard/page.tsx` có thể gọi trực tiếp hàm tổng hợp số liệu trên máy chủ rồi trả về HTML đã render, tránh việc gửi nhiều yêu cầu fetch nhỏ từ trình duyệt và rút ngắn thời gian hiển thị nội dung đầu tiên.

3.2 TypeScript và Express — Nền tảng backend

Express là khung phát triển HTTP tối giản trên nền Node.js, ra mắt từ năm 2010 và hiện vẫn nằm trong số các dự án mã nguồn mở phổ biến nhất của hệ sinh thái JavaScript [3]. Tư tưởng chủ đạo của Express là chuỗi middleware: mỗi yêu cầu đi qua một dãy hàm có chữ ký `(req, res, next)`, mỗi hàm có thể thay đổi

đối tượng yêu cầu hoặc kết thúc luồng. Điều này đặc biệt phù hợp với nhu cầu xác thực, ghi nhật ký và kiểm tra dữ liệu của PM QLKT.

TypeScript được dùng song song với Express ở toàn bộ phần backend nhằm phát hiện lỗi tại thời điểm biên dịch. Cấu hình `tsconfig.json` bật `strictNullChecks` cho lớp Service và Repository nhưng nới lỏng cho lớp Controller (do tương tác trực tiếp với `req`/`res` Express vốn có nhiều trường tuỳ chọn). Các kiểu Prisma sinh tự động từ schema được nhập vào toàn bộ tầng Service, đảm bảo mọi truy vấn cơ sở dữ liệu đều có hỗ trợ gợi ý từ trình soạn thảo.

Trong PM QLKT, đặc tính chuỗi middleware của Express được khai thác triệt để qua mẫu thiết kế năm tầng cố định: `verifyToken -> requireRole -> validate(schema) -> auditLog(options) -> controller.method`. Mỗi tầng đảm nhiệm một mối quan tâm độc lập (xác thực, phân quyền, kiểm tra dữ liệu, ghi nhật ký), cho phép tổ chức mã nguồn theo nguyên tắc đơn trách nhiệm và tái sử dụng giữa các route. Hàm phụ trợ `catchAsync` gói các handler bất đồng bộ để chuyển lỗi về middleware xử lý lỗi tập trung, loại bỏ nhu cầu viết `try/catch` lặp đi lặp lại trong từng controller.

3.3 PostgreSQL và Prisma ORM

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở với lịch sử phát triển từ năm 1986 và hiện được duy trì bởi cộng đồng PGDG [4]. Hệ này được chọn do tính ổn định cao, tuân thủ chuẩn SQL chặt chẽ và hỗ trợ nhiều kiểu dữ liệu nâng cao như JSON, JSONB, mảng, kiểu liệt kê và kiểu thời gian có múi giờ — tất cả đều được dùng trong PM QLKT.

Prisma là một ORM thế hệ mới, không sinh code dạng kiểu Active Record mà sinh ra một Client typed an toàn từ tệp khai báo `schema.prisma` duy nhất [5]. Mỗi thay đổi schema được áp dụng qua lệnh `prisma migrate dev` tạo ra các tệp SQL nằm dưới `prisma/migrations/`, có thể đẩy lên kho mã nguồn để theo dõi lịch sử. Prisma Client trả về các kiểu được suy luận từ schema, vì vậy mỗi truy vấn `findUnique`, `findMany`, `create` đều có hỗ trợ gợi ý đầy đủ ở thời điểm soạn mã.

Trong PM QLKT, schema gồm 23 model với tổng cộng 577 dòng. Quy ước đặt tên tận dụng chỉ thị `@@map("snake_case")` của Prisma để cho phép tên model trong code dùng `PascalCase` tiếng Việt (`QuanNhan`, `DanhHieuHangNam`, `HoSoCongHien`) trong khi tên bảng vật lý vẫn theo quy ước `snake_case` của PostgreSQL. Ưu điểm của tổ chức này là code đọc tự nhiên với người Việt, đồng thời giữ được khả năng truy vấn trực tiếp bằng SQL cho các nhu cầu báo cáo phức tạp ngoài Prisma. Bảng 3.1 so sánh ngắn gọn ba lựa chọn ORM phổ biến để làm rõ

lý do chọn Prisma.

Bảng 3.1: So sánh ba ORM phổ biến trên Node.js

Tiêu chí	**Sequelize**	**TypeORM**	**Prisma**
Năm ra mắt	2010	2016	2019
Kiểu định nghĩa schema	JavaScript / mô hình lớp	Decorator + lớp	DSL trong tệp `schema.prisma`
Hỗ trợ TypeScript	Có nhưng cần khai báo bổ sung	Tốt	Sinh kiểu hoàn toàn tự động
Migration	Có (kém trực quan)	Có	Sinh tự động và có thể đảo ngược
Hiệu suất truy vấn	Trung bình	Trung bình	Tối ưu nhờ engine Rust
Cộng đồng	Lớn nhưng đang chậm phát triển	Trung bình	Đang tăng nhanh
Phù hợp cho dự án có nhiều quan hệ	Cần cấu hình thủ công	Cần decorator nhiều	Khai báo gọn, sinh truy vấn join hiệu quả

Với 23 model và nhiều quan hệ N–N qua bảng trung gian (vd: `BangDeXuat <-> QuanNhan` qua `data_danh_hieu` JSONB hoặc qua `HoSoNienHan`), khả năng sinh truy vấn `include`, `select` của Prisma giúp giảm số dòng code ở tầng Repository so với việc viết câu lệnh SQL thủ công. Đây là lý do quyết định chọn Prisma cho PM QLKT.

3.4 Ant Design, Tailwind CSS và shadcn/ui — Bộ giao diện ba lớp

Frontend của PM QLKT sử dụng kết hợp ba thư viện giao diện ở các vai trò khác nhau. Ant Design là thư viện thành phần React do Alibaba phát triển [12], cung cấp các thành phần phức tạp đã hoàn thiện như `Table` có lọc và phân trang, `Form` tích hợp validation, `Select` đa lớp, `DatePicker` hỗ trợ múi giờ, `Modal` lồng nhau và `Notification`. Trong PM QLKT, Ant Design đảm nhiệm các form đề xuất nhiều bước, bảng danh sách quân nhân với cột động và các hộp thoại xác nhận phê duyệt.

Tailwind CSS là khung CSS utility-first cho phép áp các lớp đơn lẻ trực tiếp trong JSX để biểu diễn bố cục, khoảng cách, màu sắc và đáp ứng nhiều kích cỡ màn hình [13]. Tailwind không thay thế Ant Design mà bổ sung ở những vị trí cần điều chỉnh chi tiết bố cục mà các thành phần Ant Design chưa đáp ứng được, ví dụ căn lề tinh tế giữa các thẻ thông tin trên trang Bảng điều khiển, hoặc tổ chức lưới các thẻ huy chương theo hai cột trên màn hình lớn và một cột trên thiết bị di động.

Shadcn/ui có vai trò khác biệt: đây không phải là gói npm cài đặt trực tiếp mà là

một bộ sao chép mã nguồn thành phần dựa trên Radix UI. Lập trình viên dùng lệnh CLI để sao chép một thành phần (vd: `button`, `dropdown-menu`, `dialog`) vào thư mục `components/ui/` của dự án và sửa lại tự do. Cách tiếp cận này tránh tình trạng phụ thuộc vào phiên bản thư viện cứng và đặc biệt hữu ích cho các thành phần ít gặp như danh sách lệnh tổ hợp phím trong khu vực dành cho lập trình viên (DevZone) của hệ thống. Quy tắc sử dụng giữa ba thư viện được đồng thuận trong nhóm: Ant Design ưu tiên cho biểu mẫu và bảng nghiệp vụ, Tailwind cho bố cục và chỉnh sửa nhẹ, shadcn/ui cho các thành phần đặc thù cần kiểm soát trực tiếp mã nguồn.

3.5 Socket.IO — Kênh thông báo thời gian thực

Socket.IO là thư viện trừu tượng hoá giao thức WebSocket, hỗ trợ tự động chuyển sang giao thức HTTP long polling khi mạng giữa máy khách và máy chủ không cho phép kết nối WebSocket trực tiếp [7]. Thư viện cung cấp khái niệm "phòng" (room) cho phép phát thông điệp tới một nhóm kết nối được đặt nhãn, rất phù hợp với mô hình thông báo theo người dùng hoặc theo đơn vị.

Trong PM QLKT, Socket.IO được dùng cho hai trường hợp cụ thể. Thứ nhất, khi một cán bộ ADMIN nhập một tệp Excel chứa hàng trăm bản ghi danh hiệu, máy chủ phát các sự kiện tiến độ theo từng phần trăm hoàn thành để giao diện cập nhật thanh tiến trình mà không phải thiết lập polling phía trình duyệt. Thứ hai, khi một đề xuất được phê duyệt, máy chủ phát thông điệp tới tất cả các kết nối thuộc vai trò MANAGER của đơn vị có quân nhân được duyệt; giao diện hiển thị thông báo xuất hiện ở góc màn hình mà không cần làm mới trang. Các kết nối được xác thực qua JWT lấy từ `localStorage` và truyền vào tuỳ chọn `auth: { token }` của Socket.IO trong handshake; máy chủ dùng cùng `JWT_SECRET` với REST để xác minh, nhờ vậy logic xác thực không bị phân nhánh giữa REST và WebSocket.

3.6 JSON Web Token với cơ chế làm mới luân phiên

Hệ thống áp dụng JWT theo chuẩn RFC 7519 [6] cho cơ chế xác thực không trạng thái. Mỗi phiên đăng nhập tạo ra hai token: Access Token có thời hạn ngắn (30 phút) ký bằng `JWT_SECRET`, chứa định danh người dùng, mã vai trò và liên kết tới quân nhân, được gửi kèm mỗi yêu cầu HTTP qua header `Authorization: Bearer`; Refresh Token có thời hạn dài hơn (7 ngày) ký bằng `JWT_REFRESH_SECRET` riêng, chỉ dùng để cấp Access Token mới khi token hiện tại hết hạn. Frontend lưu cả hai token vào `localStorage`, còn backend lưu thêm Refresh Token vào cột `TaiKhoan.refreshToken` để có khả năng thu hồi từ phía máy chủ.

Cơ chế làm mới luân phiên (refresh token rotation) yêu cầu mỗi lần dùng Refresh

Token, máy chủ phát hành đồng thời cặp Access + Refresh Token mới và ghi đè cột `refreshToken` trong cơ sở dữ liệu — mọi lần dùng Refresh Token cũ sau đó đều bị từ chối do không còn khớp với giá trị trong cơ sở dữ liệu. Cách làm này hạn chế nguy cơ một Refresh Token bị đánh cắp vẫn dùng được lâu dài: nếu kẻ tấn công và người dùng hợp pháp cùng dùng một Refresh Token thì lần làm mới thứ hai sẽ thất bại, đồng thời cho phép buộc đăng xuất từ phía máy chủ bằng cách xoá cột `refreshToken`. Việc dùng hai khoá bí mật riêng biệt (defense in depth) đảm bảo nếu một khoá bị lộ thì chỉ một loại token bị compromise. Mật khẩu trước khi lưu vào bảng `TaiKhoan` được băm bằng bcrypt với hệ số chi phí 10 [9], đảm bảo cân bằng giữa thời gian xử lý đăng nhập (vào khoảng 80 ms trên máy chủ phát triển) và độ khó tấn công vét cạn nếu cơ sở dữ liệu bị rò rỉ.

3.7 Joi và Zod — Kiểm tra dữ liệu ở hai phía

PM QLKT chia kiểm tra dữ liệu thành hai lớp với hai thư viện khác nhau, mỗi thư viện phù hợp với đặc thù của lớp tương ứng. Tại backend, mọi route nhận dữ liệu từ client đều đi qua middleware `validate(schema)` dùng Joi để kiểm tra `req.body`, `req.query` và `req.params` [11]. Joi cung cấp API định nghĩa schema theo phong cách lập trình hàm nối tiếp, gắn được thông điệp tiếng Việt cho từng lỗi và hỗ trợ tuỳ chọn `stripUnknown: true` để loại bỏ các trường không khai báo trong schema — tránh việc client lén đẩy lên các trường ngoài ý muốn.

Tại frontend, các form do Ant Design Form quản lý được xác thực bằng Zod thông qua thư viện trung gian [10]. Zod có ưu điểm là tích hợp chặt với TypeScript: kiểu của giá trị form được suy luận trực tiếp từ schema (`type FormValues = z.infer<typeof formSchema>`), tránh việc duy trì hai khai báo song song giữa interface và schema. Việc sử dụng hai thư viện khác nhau ở hai phía thoạt nhìn có vẻ trùng lặp nhưng đáp ứng đúng vai trò: Zod tối ưu cho trải nghiệm soạn thảo trên frontend, còn Joi tích hợp tốt với chuỗi middleware Express ở backend. Quan trọng là logic xác thực không tin tưởng vào lớp frontend — backend luôn kiểm tra lại dữ liệu trước khi ghi vào cơ sở dữ liệu.

3.8 Jest — Khung kiểm thử đơn vị

Jest là khung kiểm thử do Meta phát triển [8], có ưu điểm là cấu hình tối thiểu, hỗ trợ chế độ theo dõi (watch mode) chạy lại các bộ kiểm thử bị ảnh hưởng khi tệp nguồn thay đổi và có hệ thống trình giả (mock) tích hợp sẵn. Phiên bản dùng cho PM QLKT là Jest 29 kết hợp với `ts-jest` để chạy trực tiếp các tệp TypeScript mà không cần biên dịch trước.

Tại thời điểm hoàn thiện đồ án, kho kiểm thử của PM QLKT gồm 75 bộ kiểm thử với 890 ca kiểm thử, tổ chức theo các thư mục

`tests/services/`, `tests/scenarios/`, `tests/approve/`, `tests/submit/`, `tests/import/`, `tests/authz/`. Hai nhóm trọng yếu nhất là `eligibility-bkbqp-personal.test.ts`, `eligibility-cstdtq-personal.test.ts`, `eligibility-bkttcp-personal.test.ts` (cùng các phiên bản đơn vị) tập trung vào các kịch bản đa dạng của rule chuỗi: chu kỳ vừa đủ, chu kỳ thừa, lỡ đợt, BKBTBQP rơi ra ngoài cửa sổ trượt 3 năm, và trường hợp đặc biệt khi quân nhân chưa nhận đủ NCKH. Các trường hợp này được cố định trước trong tệp `errorMessages.ts` để đảm bảo bất kỳ sửa đổi nào trong logic chuỗi đều phải cập nhật song song giá trị mong đợi của ca kiểm thử, giúp tránh tình trạng test "trôi" theo code.

3.9 ExcelJS — Công cụ nhập và xuất Excel

ExcelJS là thư viện JavaScript cho phép đọc, ghi và thao tác workbook định dạng `.xlsx` ở phía máy chủ Node.js [14]. Khác với các thư viện chỉ tạo CSV hoặc HTML giả lập Excel, ExcelJS hỗ trợ định dạng ô (font, màu nền, viền), khoá ô để tránh người dùng chỉnh sửa các cột hệ thống, công thức và đặc biệt là Data Validation rule cho phép tạo các danh sách thả xuống ngay trong tệp mẫu xuất ra.

Trong PM QLKT, ExcelJS phục vụ hai mục đích đối xứng. Khi xuất tệp mẫu nhập liệu cho từng nhóm khen thưởng, máy chủ sinh workbook với các sheet riêng (`QuanNhan`, `DanhHieuHangNam`, `ThanhTichKhoaHoc`, ...), cài đặt cột cố định, áp Data Validation lên cột danh hiệu để chỉ chấp nhận các giá trị hợp lệ và đính kèm chú thích tiếng Việt giải thích từng cột. Khi nhập tệp Excel do người dùng tải lên, máy chủ đọc tuần tự từng dòng, đối chiếu với schema Joi và Prisma rồi đưa kết quả vào màn hình xem trước trước khi đưa vào cơ sở dữ liệu. Quy trình hai bước (xem trước rồi xác nhận) tận dụng đặc tính giao dịch của Prisma ở bước xác nhận để đảm bảo nguyên tử tính: nếu có bất kỳ dòng nào lỗi vào thời điểm ghi, toàn bộ thao tác sẽ được huỷ bỏ.

Kết chương. Chương này đã giới thiệu chín nhóm công nghệ trọng yếu của PM QLKT cùng lý do lựa chọn từng nhóm trong ngữ cảnh đặc thù của đề tài. Frontend dùng Next.js 14 App Router cho phép server-side rendering theo từng vai trò; backend là Express + TypeScript theo kiến trúc middleware nối tiếp; cơ sở dữ liệu PostgreSQL truy cập qua Prisma ORM với 23 model. Các thư viện phụ trợ (Ant Design, Tailwind CSS, shadcn/ui, Socket.IO, JWT, Joi, Zod, Jest, ExcelJS) bổ sung các năng lực cụ thể cho biểu mẫu, thông báo realtime, xác thực hai phía, kiểm thử và xử lý Excel. Toàn bộ nền tảng công nghệ này đặt cơ sở cho phần thiết kế kiến trúc và triển khai chi tiết được trình bày ở chương tiếp theo.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày toàn bộ quá trình từ thiết kế kiến trúc đến triển khai sản phẩm. Mở đầu là phần mô tả lựa chọn kiến trúc tổng thể và các sơ đồ thiết kế chi tiết. Tiếp đó là kết quả xây dựng kèm các đoạn mã minh hoạ lấy trực tiếp từ kho mã nguồn của dự án. Khép lại chương là báo cáo kết quả kiểm thử và hướng dẫn triển khai.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống áp dụng **kiến trúc phân tầng (layered architecture)** với sáu lớp xếp chồng từ ngoài vào trong: Route → Middleware → Controller → Service → Repository → Prisma Client. Mỗi lớp chỉ tương tác trực tiếp với lớp liền kề, không gọi nhảy lớp. Đây là biến thể mở rộng của mô hình MVC truyền thống, trong đó Service và Repository được tách riêng để rõ ràng hoá ranh giới giữa logic nghiệp vụ và truy cập dữ liệu.

Việc tách lớp Repository khỏi lớp Service được áp dụng từ một lần tái cấu trúc lớn của dự án. Trước đó, Service gọi `prisma.danhHieuHangNam.findMany(...)` trực tiếp, dẫn tới việc viết kiểm thử đơn vị cho Service phải mock toàn bộ Prisma Client — phức tạp và dễ tạo ra mock không chính xác. Sau khi tách, Service gọi qua giao diện `danhHieuHangNamRepository.findMany(...)`; kiểm thử đơn vị cho Service chỉ cần mock các phương thức của Repository, mã kiểm thử đơn giản hơn nhiều và phản ánh đúng hợp đồng giữa hai lớp.

Lý do không chọn các kiến trúc khác:

- **Pure MVC** thiếu khái niệm Repository, dẫn tới các hàm Service vừa làm logic nghiệp vụ vừa làm truy vấn — vi phạm nguyên tắc đơn trách nhiệm đối với một dự án có nhiều quan hệ phức tạp như PM QLKT (23 model với nhiều quan hệ N–N).
- **Clean Architecture / Hexagonal** đầy đủ với Use Case Layer, Entity Layer, Adapter Layer là quá phức tạp cho quy mô dự án. Lớp Use Case sẽ trùng lặp gần như hoàn toàn với lớp Service trong layered architecture.
- **Microservices** không phù hợp với môi trường mạng nội bộ một máy chủ; chia tách dịch vụ làm tăng độ phức tạp triển khai mà không mang lại lợi ích về quy mô tải.

Hình 4.1: Sơ đồ kiến trúc phân tầng sáu lớp

4.1.2 Tổng quan hệ thống

Sản phẩm được tổ chức thành ba thành phần độc lập triển khai:

- **Frontend (FE-QLKT)**: ứng dụng Next.js 14 chạy bằng Node.js. Cung cấp giao diện cho cả bốn vai trò người dùng. Kết nối tới backend qua REST API (HTTP) và Socket.IO (WebSocket).
- **Backend (BE-QLKT)**: API server Express + TypeScript chạy bằng Node.js. Cung cấp REST endpoint, Socket.IO server, thực hiện toàn bộ logic nghiệp vụ và quản lý phiên đăng nhập.
- **Cơ sở dữ liệu**: PostgreSQL chạy trong tiến trình riêng. Backend kết nối qua Prisma Client, không cho phép FE truy cập trực tiếp.

Cả ba thành phần này được triển khai trên cùng một máy chủ Linux của Học viện và giao tiếp qua localhost. Bên ngoài, người dùng truy cập qua trình duyệt thông qua mạng LAN; FE phục vụ tệp tĩnh (HTML, CSS, JS đã được biên dịch), gọi tới BE qua proxy. Sơ đồ Hình 4.2 thể hiện luồng này.

Hình 4.2: Sơ đồ kiến trúc tổng quan FE — BE — DB

4.1.3 Thiết kế tổng quan các gói

Cấu trúc thư mục mã nguồn được tổ chức theo nguyên tắc đơn trách nhiệm và đặt tên theo tiếng Anh chuẩn để dễ tham chiếu trong các sơ đồ.

a, Gói phía frontend

Mã nguồn frontend được tổ chức bên trong thư mục `FE-QLKT/src/`. Gói `app/` chứa cây định tuyến của Next.js App Router với bốn nhánh tương ứng bốn vai trò người dùng (super-admin, admin, manager, user). Gói `components/` chứa các thành phần React dùng chung, được tách thành các thư mục con theo miền nghiệp vụ: `auth/` cho màn hình đăng nhập và đổi mật khẩu, `proposals/` cho biểu mẫu tạo đề xuất nhiều bước với mỗi loại trong bảy loại đề xuất, `personnel/` cho danh sách và chi tiết quân nhân, `system-logs/` cho bảng nhật ký kiểm toán. Gói `contexts/` đặt `AuthContext` quản lý trạng thái đăng nhập trong toàn ứng dụng. Gói `hooks/` chứa các React hook tự viết như `useFetch`, `useAuthGuard`, `useSocket`. Gói `lib/` chứa các module phụ trợ: `api/` chia theo từng miền nghiệp vụ và đóng gói các yêu cầu REST tới backend, `award/` chứa các hàm trợ giúp render chuỗi danh hiệu phía frontend, `schemas.ts` định

nghĩa các schema Zod xác thực biểu mẫu. Gói `constants/` chứa các hằng số dùng chung như danh sách vai trò, trạng thái, danh hiệu.

Hình 4.3: Sơ đồ gói phía frontend

b, Gói phía backend

Mã nguồn backend được tổ chức bên trong thư mục `BE-QLKT/src/` theo sáu lớp đã trình bày ở mục 4.1.1. Gói `routes/` định nghĩa các điểm cuối REST API, mỗi miền nghiệp vụ một tệp. Gói `middlewares/` chứa các tệp `auth.ts` (xác thực và phân quyền), `auditLog.ts` (ghi nhật ký mọi thao tác mutate), `unitFilter.ts` (lọc dữ liệu theo phạm vi đơn vị của Manager) và `validate.ts` (bọc Joi validation). Gói `controllers/` chứa các tệp điều hướng yêu cầu HTTP từ route tới service, được giữ mỏng (mỗi phương thức dưới 50 dòng mã). Gói `services/` chứa logic nghiệp vụ chính, được tách tiếp thành các thư mục con: `proposal/strategies/` cho strategy registry của bảy loại đề xuất, `eligibility/` cho hàm `chainEligibility` xét rule chuỗi cá nhân và đơn vị, `profile/annual.ts` cho logic chuỗi cá nhân và `profile/unit.ts` cho logic chuỗi đơn vị. Gói `repositories/` đóng gói toàn bộ truy cập Prisma — mỗi model một tệp `.repository.ts`. Gói `helpers/` chứa các tiện ích phụ trợ chia theo nhóm: `auditLog/`, `notification/`, `excel/`, `awardValidation/`. Gói `validations/` chứa các schema Joi cho từng route. Tệp `prisma/schema.prisma` ở cấp gốc của module backend định nghĩa toàn bộ 23 model.

Hình 4.4: Sơ đồ gói phía backend

4.1.4 Thiết kế chi tiết gói nghiệp vụ Khen thưởng cá nhân hằng năm

Để minh hoạ cách các tầng phối hợp với nhau trong một module nghiệp vụ cụ thể, phần này trình bày chi tiết module xử lý chuỗi danh hiệu hằng năm cá nhân — module phức tạp nhất của hệ thống.

Khi người dùng truy cập trang hồ sơ một quân nhân, dòng chảy dữ liệu diễn ra như sau:

1. **Route** `GET /api/personnel/:id/profile` (định nghĩa tại `routes/profile.route.ts`) tiếp nhận yêu cầu và đẩy qua chuỗi middleware `verifyToken -> requireAuth -> unitFilter`.
2. **Controller** `profileController.getAnnualProfile` (tại `con-`

trollers/profile.controller.ts) đọc tham số personnelId, gọi service tương ứng, đóng gói kết quả vào ResponseHelper.success.

3. **Service** profileService.recalculateAnnualProfile (tại services/profile/annual.ts) thực thi logic gồm: nạp danh sách DanhHieuHangNam qua repository, gọi computeChainContext để dẫn xuất ngữ cảnh chuỗi, lặp qua PERSONAL_CHAIN_AWARDS và gọi checkChainEligibility cho mỗi tier, ghi kết quả vào bảng suy diễn HoSoHangNam.
4. **Eligibility module** services/eligibility/chainEligibility.ts chứa hàm checkChainEligibility(config, context, hasReceivedLifetime) — hàm thuần (pure function), không truy vấn cơ sở dữ liệu, dễ kiểm thử.
5. **Repository** repositories/danhHieuHangNam.repository.ts cung cấp các phương thức findManyByPersonnelId, findUniqueByPersonnelYearAward.
6. **Prisma Client** thực thi truy vấn SQL tới bảng danh_hieu_hang_nam.

Sự tách biệt giữa Service (logic nghiệp vụ) và Eligibility module (rule thuần) là chủ ý: Eligibility module có thể được kiểm thử với 100 % phủ rule mà không cần mock cơ sở dữ liệu. Service chỉ phụ trách điều phối — nạp dữ liệu, gọi rule, ghi kết quả — nên cũng dễ kiểm thử với mock Repository.

Hình 4.5: Sơ đồ gói chi tiết module khen thưởng cá nhân hằng năm

4.1.5 Thiết kế lớp

Phần này trình bày sơ đồ lớp cho năm module nghiệp vụ chính. Mỗi sơ đồ chỉ ra các lớp/interface cốt lõi cùng các phương thức quan trọng và mối quan hệ.

Module xử lý chuỗi danh hiệu. Trung tâm là interface ChainAwardConfig chứa cấu hình một tier chuỗi (mã danh hiệu, số năm chu kỳ, danh sách cờ tiền điều kiện, có yêu cầu NCKH hay không, có phải lifetime hay không, tên cột cờ trong cơ sở dữ liệu, nhãn chuỗi). Hai mảng cấu hình PERSONAL_CHAIN_AWARDS và UNIT_CHAIN_AWARDS chứa danh sách các tier tương ứng cho cá nhân và đơn vị. Hàm checkChainEligibility(config, context, hasReceivedLifetime, flagsInWindow) đọc cấu hình và đối chiếu với ngữ cảnh thực tế để trả về kết quả EligibilityResult gồm hai trường eligible: boolean và reason: string. Interface ChainContext đóng gói

các chỉ số dẫn xuất từ lịch sử (độ dài chuỗi, năm gần nhất nhận từng tier, số cờ trong cửa sổ trượt).

Module Strategy đề xuất. Interface `ProposalStrategy` định nghĩa bốn phương thức bắt buộc cho mỗi loại đề xuất: `buildSubmitPayload`, `validateApprove`, `importInTransaction`, `buildSuccessMessage`. Bảy lớp triển khai (`caNhanHangNamStrategy`, `donViHangNamStrategy`, `nienHanStrategy`, `hcQkqtStrategy`, `kncStrategy`, `congHienStrategy`, `nckhStrategy`) đăng ký vào `REGISTRY` trong `services/proposal/strategies/index.ts`. Hai lớp single-medal (`hcQkqtStrategy` và `kncStrategy`) chia sẻ logic qua hàm trợ giúp `singleMedalImporter` để tránh trùng lặp.

Module xác thực và phân quyền. Lớp chính là `AuthService` cung cấp các phương thức `login`, `refresh`, `logout`, `changePassword`. Middleware `verifyToken` đọc cookie, xác minh Access Token, gắn `req.user`. Middleware `requireRole(...allowed)` factory trả về middleware kiểm tra `req.user.role` thuộc danh sách cho phép. Middleware `unitFilter` truy vấn `co_quan_don_vi_id`/`don_vi_truc_thuoc_id` của người dùng và áp giới hạn phạm vi cho mỗi yêu cầu của Manager.

Module quản lý đề xuất. Lớp `ProposalService` điều phối luồng tạo và phê duyệt đề xuất, dùng `proposalRepository` để truy vấn và cập nhật. Lớp `ApproveOrchestrator` (tại `services/proposal/approve/`) thực hiện logic phê duyệt phức tạp, gọi tuần tự `validation.ts -> decisionMappings.ts -> import.ts` trong cùng một transaction.

Module nhập – xuất Excel. Bao gồm các hàm trợ giúp dùng chung (`loadWorkbook`, `getAndValidateWorksheet`, `parseHeaderMap`, `sanitizeRowData`) và các strategy chuyên biệt cho từng loại nghiệp vụ. Mỗi strategy có hai phương thức `previewImport` (chỉ kiểm tra) và `confirmImport` (mở transaction ghi).

Hình 4.6: Sơ đồ lớp năm module nghiệp vụ trọng yếu

4.1.6 Biểu đồ tuần tự

Bảy luồng nghiệp vụ quan trọng nhất được mô hình hoá bằng sơ đồ tuần tự (sequence diagram). Mỗi sơ đồ thể hiện sự tương tác giữa các đối tượng theo thời gian.

Sequence 4.1 — Đăng nhập với refresh token rotation. Người dùng

nhập tên đăng nhập và mật khẩu → FE gọi `POST /auth/login` → `authController.login` → `authService.login` → `accountRepository.findByUsername` → `bcrypt.compare` → tạo Access + Refresh Token → lưu Refresh Token vào cột `TaiKhoan.refreshToken` → ghi `SystemLog` → trả cặp token cho FE để ghi vào `localStorage`.

Sequence 4.2 — Tạo đề xuất. Manager hoàn thành ba bước form → FE gọi `POST /api/proposals` → middleware chain → `proposalService.submitProposal` → `getProposalStrategy(type).buildSubmitPayload` → `proposalRepository.create` → phát Socket.IO event tới Admin → trả về đề xuất đã tạo.

Sequence 4.3 — Phê duyệt đề xuất. Admin mở chi tiết → bấm "Phê duyệt" → FE gọi `POST /api/proposals/:id/approve` → middleware chain → `approveOrchestrator.run` → mở transaction Prisma → `validation.preflight` → `strategy.validateApprove` → `decisionMappings.attach` → `strategy.importInTransaction` → cập nhật `BangDeXuat.status = APPROVED` → tính lại hồ sơ suy diễn → commit → `auditLog` → phát Socket.IO event.

Sequence 4.4 — Tính lại điều kiện chuỗi cho một quân nhân. FE gọi `GET /api/personnel/:id/annual-profile` → `profileController.getAnnualProfile` → `profileService.recalculateAnnualProfile` → `danhHieuHangNamRepository.findManyByPersonnelId` → `computeChainContext(rows, currentYear)` → vòng for qua `PERSONAL_CHAIN_AWARDS` gọi `checkChainEligibility` → `hoSoHangNamRepository.upsert` → trả kết quả.

Sequence 4.5 — Nhập Excel hai bước. Admin tải tệp → FE gọi `POST /api/annual-rewards/import/preview` → `excelHelper.loadWorkbook` → đọc từng trang tính → cho mỗi dòng: chạy schema Joi và rule nghiệp vụ → trả về `{valid: [...], errors: [...]}`. Admin xác nhận → FE gọi `POST /api/annual-rewards/import/confirm` → mở transaction → ghi tuần tự → commit hoặc rollback.

Sequence 4.6 — Sao lưu định kỳ tự động. Cron job nội bộ kích hoạt mỗi 24 giờ → `backupService.runScheduledBackup` → spawn child process `pg_dump` → ghi tệp SQL vào `backups/YYYY-MM-DD_HH-mm-ss.sql` → ghi nhật ký `action = CREATE, resource = backup`.

Sequence 4.7 — Thông báo realtime sau phê duyệt. Sau khi Admin phê duyệt,

`notificationService.notifyApproval` được gọi → tra cứu các quân nhân đối tượng → tạo bản ghi `ThongBao` → emit Socket.IO event tới các phòng tương ứng (`user:<id>`, `unit:<id>`) → FE người nhận hiển thị toast notification.

Hình 4.7: Bảy sơ đồ tuần tự cho các luồng nghiệp vụ chính

4.1.7 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu được mô hình hoá thành 23 model trong tệp `prisma/schema.prisma`. Hình 4.8 là sơ đồ ERD tổng thể; sáu sơ đồ ERD phân module phóng to từng nhóm bảng có liên quan trực tiếp với nhau.

Hình 4.8: Sơ đồ ERD tổng thể của 23 model

Sơ đồ ERD tổng thể bao quát toàn bộ 23 bảng với các quan hệ khoá ngoại chính. Có thể nhận diện ba "trục" chính: (1) trục tổ chức gồm `CoQuanDonVi`, `DonViTrucThuoc`, `ChucVu`, `LichSuChucVu`; (2) trục quân nhân gồm `QuanNhan`, `TaiKhoan`; (3) trục khen thưởng gồm `BangDeXuat`, `DanhHieuHangNam`, `KhenThuongHCCSVV`, `HuanChuongQuanKyQuyetThang`, `KyNiemChuongVSNXDQDNDVN`, `KhenThuongHCBVTQ`, `KhenThuongDotXuat`, `ThanhTichKhoaHoc`. Các bảng suy diễn `HoSoHangNam`, `HoSoNienHan`, `HoSoCongHien`, `HoSoDonViHangNam` chứa kết quả tính toán lại được tự động sau mỗi thay đổi nguồn.

Sáu bảng mô tả chi tiết schema sau đây được chọn vì là các bảng trọng yếu nhất, sử dụng nhiều cấu trúc dữ liệu phức tạp (JSON, mảng, kiểu thời gian).

Bảng 4.1: Schema bảng `QuanNhan` (Personnel)

Cột	Kiểu	Ràng buộc	Mô tả
`id`	String (CUID)	PK	Khoá chính
`cccd`	String(12)	UNIQUE, NOT NULL	Số căn cước công dân
`ho_ten`	String	NOT NULL	Họ và tên đầy đủ
`ngay_sinh`	DateTime	NULL	Ngày sinh
`gioi_tinh`	Enum(NAM, NU)	NOT NULL	Giới tính
`ngay_nhap_ngu`	DateTime	NULL	Ngày nhập ngũ
`ngay_xuat_ngu`	DateTime	NULL	Ngày xuất ngũ (nếu đã rời)
`cap_bac`	String	NULL	Cấp bậc hiện tại
`co_quan_don_vi_id`	String	FK → CoQuanDonVi	Liên kết tới CQĐV
`don_vi_truc_thuoc_id`	String	FK → DonViTrucThuoc	Liên kết tới ĐVTT
`chuc_vu_id`	String	FK → ChucVu	Chức vụ hiện tại
`createdAt`	Timestamptz	DEFAULT now()	Thời điểm tạo

Ràng buộc nghiệp vụ: chỉ một trong `co_quan_don_vi_id` và `don_vi_truc_thuoc_id` được điền. Khi cập nhật chuyển đơn vị, cần dùng transaction để cập nhật cả bộ đếm `so_luong` của hai đơn vị liên quan.

Bảng 4.2: Schema bảng `BangDeXuat` (Proposal)

Cột	Kiểu	Ràng buộc	Mô tả
`id`	String (CUID)	PK	
`loai_de_xuat`	String	NOT NULL	Một trong 7 mã loại đề xuất
`nguoi_de_xuat_id`	String	FK → TaiKhoan	Người tạo đề xuất
`nam`	Int	NOT NULL	Năm xét
`thang`	Int	NULL	Tháng xét (chỉ với một số loại)
`status`	Enum	NOT NULL	PENDING, APPROVED, REJECTED
`data_danh_hieu`	JSON	NULL	Dữ liệu chi tiết hằng năm
`data_thanh_tich`	JSON	NULL	Dữ liệu thành tích NCKH
`data_nien_han`	JSON	NULL	Dữ liệu niên hạn
`data_cong_hien`	JSON	NULL	Dữ liệu cống hiến
`files_attached`	JSON	NULL	Mảng file đính kèm

Việc dùng JSON cho 4 trường `data_*` là chủ ý. Mỗi loại đề xuất có schema riêng nên việc tạo nhiều bảng trung gian sẽ phá vỡ tính đồng nhất của bảng `BangDeXuat`. JSON đủ để đảm bảo schema linh hoạt; xác thực schema được đảm nhiệm ở tầng Joi và `ProposalStrategy`.

Bảng 4.3: Schema bảng `DanhHieuHangNam` (Annual Award)

Cột	Kiểu	Ràng buộc	Mô tả
`id`	String (CUID)	PK	
`quan_nhan_id`	String	FK → QuanNhan	
`nam`	Int	UNIQUE(quan_nhan_id, nam)	Năm danh hiệu
`danh_hieu`	String	NULL	Mã danh hiệu cơ bản
`nhan_bkbqp`	Boolean	DEFAULT false	Cờ nhận BKBTBQP
`so _ quyet _ dinh _ bkbqp`	String	NULL	Số quyết định BKBTBQP
`nhan_cstdtq`	Boolean	DEFAULT false	Cờ nhận CSTĐTQ
`so _ quyet _ dinh _ cstdtq`	String	NULL	
`nhan_bkttcp`	Boolean	DEFAULT false	Cờ nhận BKTTCP
`so _ quyet _ dinh _ bkttcp`	String	NULL	
`ghi_chu`	String	NULL	

Ràng buộc duy nhất `(quan_nhan_id, nam)` đảm bảo mỗi quân nhân chỉ có một bản ghi cho một năm. Các cờ `nhan_*` cho phép cộng dồn các tier chuỗi vào cùng năm.

Bảng 4.4: Schema bảng `LichSuChucVu` (Position History)

Cột	Kiểu	Ràng buộc	Mô tả
`id`	String (CUID)	PK	
`quan_nhan_id`	String	FK → QuanNhan	
`chuc_vu_id`	String	FK → ChucVu	
`ngay_bat_dau`	DateTime	NULL	Ngày bắt đầu giữ chức vụ
`ngay_ket_thuc`	DateTime	NULL	NULL = đang giữ
`he_so_chuc_vu`	Decimal(3,2)	NOT NULL	Hệ số chức vụ
`so_thang`	Int	NOT NULL	Số tháng giữ chức vụ

Trường `so_thang` được tự động cập nhật bởi hàm `recalcPosition-`

`Months` mỗi khi có thay đổi mốc thời gian.

Bảng 4.5: Schema bảng `SystemLog` (Audit Log)

Cột	Kiểu	Ràng buộc	Mô tả
`id`	String (CUID)	PK	
`nguoi_thuc_hien_id`	String	FK → TaiKhoan, NULL	Có thể NULL trước đăng nhập
`nguoi_thuc_hien_role`	String	NOT NULL	Snapshot vai trò
`action`	String	NOT NULL	CREATE, UPDATE, DELETE, ...
`resource`	String	NOT NULL	Loại tài nguyên
`resource_id`	String	NULL	Mã định danh tài nguyên
`description`	String	NOT NULL	Mô tả tiếng Việt
`payload`	JSON	NULL	Trạng thái trước–sau
`createdAt`	Timestamptz	DEFAULT now()	

Bảng 4.6: Schema bảng `HoSoHangNam` (Annual Profile — Derived)

Cột	Kiểu	Ràng buộc	Mô tả
`id`	String (CUID)	PK	
`quan_nhan_id`	String	UNIQUE FK → QuanNhan	Mỗi quân nhân một bản ghi
`cstdcs_lien_tuc`	Int	NOT NULL	Độ dài chuỗi CSTĐCS
`du_dieu_kien_bkbqp`	Boolean	DEFAULT false	Cờ đủ điều kiện BKBT-BQP
`du_dieu_kien_cstdtq`	Boolean	DEFAULT false	
`du_dieu_kien_bkttcp`	Boolean	DEFAULT false	
`goi_y`	String	NULL	Thông điệp gợi ý tiếng Việt
`last_recalc_at`	Timestamptz	DEFAULT now()	Thời điểm tính lại gần nhất

Toàn văn schema (618 dòng, 23 model) được tổ chức trong tệp `BE-QLKT/prisma/schema.prisma` của kho mã nguồn dự án.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Giao diện hệ thống được thiết kế trên cơ sở wireframe ASCII đơn giản trước khi triển khai bằng React + Ant Design. Sáu wireframe chính thể hiện bố cục các trang trọng tâm.

Trang chủ Admin có cấu trúc ba khu vực: thanh điều hướng bên trái (cây CQĐV — ĐVTT có thể thu gọn), khu vực dashboard ở giữa với bốn thẻ thống kê (tổng quân nhân, tổng đơn vị, đề xuất chờ duyệt, khen thưởng tháng này), khu vực phụ bên phải với danh sách thông báo realtime. Bố cục ba khu vực này được giữ nhất quán xuyên suốt các trang nghiệp vụ chính (Quản lý quân nhân, Quản lý đề xuất, Quản lý quyết định) để giảm chi phí học thuộc cho người dùng mới.

Trang chi tiết quân nhân chia thành các tab tương ứng các nhóm khen thưởng: Hằng năm, Niên hạn, Cống hiến, Quân kỳ, Kỷ niệm chương, NCKH, Đột xuất. Mỗi tab có bảng dữ liệu kèm thanh tiến trình hiển thị các chu kỳ đang đến mốc đề nghị (vd: "BKBTBQP: 1/2 năm CSTĐCS đã tích luỹ"). Cách tiếp cận trực quan này giúp Manager nhanh chóng nhận biết quân nhân nào sắp đủ điều kiện.

Form tạo đề xuất áp dụng mẫu nhiều bước (Multi-Step Wizard) ba bước. Bước 1: chọn loại đề xuất (radio button có icon đại diện cho từng nhóm) và năm/tháng. Bước 2: chọn quân nhân hoặc đơn vị từ bảng có lọc theo nhiều tiêu chí. Bước 3: thiết lập danh hiệu cụ thể cho từng đối tượng đã chọn, kèm gợi ý từ hệ thống. Người dùng có thể quay lại các bước trước mà không mất dữ liệu nhờ trạng thái được lưu ở Context.

Trang nhập Excel chia hai phần: trên là vùng kéo thả tệp + nút tải xuống tệp mẫu; dưới là bảng xem trước với hai tab "Hợp lệ" và "Lỗi" — tab Lỗi tô đỏ và liệt kê chỉ số dòng kèm mô tả cụ thể. Nút "Xác nhận nhập" chỉ kích hoạt khi có ít nhất một dòng hợp lệ.

Trang nhật ký hệ thống dạng bảng có nhóm theo ngày. Bộ lọc ở đầu trang cho phép chọn vai trò người thực hiện, loại tài nguyên, hành động, khoảng thời gian. Mỗi dòng có thể mở rộng để xem `payload` JSON chi tiết.

4.2.2 Hệ thống thiết kế (Design System)

Bảng màu của hệ thống kế thừa mặc định của Ant Design 5 (gam xanh đại dương cho hành động chính, đỏ cho hành động xoá, vàng cho cảnh báo, xanh lá cho thành công) với một số tinh chỉnh nhỏ qua Tailwind CSS để phù hợp đặc thù môi trường quân sự (giảm độ bão hoà của các màu phụ để giao diện trông trang trọng hơn).

Hệ chữ chính dùng phông `Inter` (sans-serif) cho phần thân nội dung do khả

năng hiển thị tốt với tiếng Việt có dấu, và phông `Source Code Pro` cho khu vực hiển thị mã quyết định và mã định danh kỹ thuật (CCCD, mã quyết định) để dễ đọc các chuỗi số dài.

Khoảng cách (spacing) được chuẩn hoá theo thang 4 px của Tailwind. Tất cả các thẻ (Card) có lề trong tối thiểu 16 px để tránh nội dung sát mép. Bán kính bo góc được chọn 6 px cho mọi phần tử để tạo cảm giác hiện đại nhưng vẫn nghiêm túc.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Sản phẩm sử dụng tổng cộng 30 thư viện mã nguồn mở chia làm ba nhóm chức năng. Bảng 4.7 liệt kê các thư viện chính.

Bảng 4.7: Thư viện và công cụ chính

STT	Mục đích	Thư viện / Công cụ	Phiên bản
1	Khung phát triển frontend	Next.js	14.x
2	Thư viện thành phần UI	Ant Design	5.x
3	Khung CSS utility	Tailwind CSS	3.x
4	Bộ thành phần đặc thù	shadcn/ui (Radix UI)	1.x
5	Khung phát triển backend	Express	4.x
6	Ngôn ngữ kiểu tĩnh	TypeScript	5.x
7	Hệ quản trị CSDL	PostgreSQL	15
8	ORM	Prisma	5.x
9	Xác thực JWT	jsonwebtoken	9.x
10	Băm mật khẩu	bcrypt	5.x
11	Kiểm tra dữ liệu BE	Joi	17.x
12	Kiểm tra dữ liệu FE	Zod	3.x
13	Thông báo realtime	Socket.IO	4.x
14	Đọc/ghi Excel	ExcelJS	4.x
15	Khung kiểm thử	Jest + ts-jest	29.x
16	Quản lý tiến trình production	PM2	5.x
17	Header bảo mật HTTP	Helmet	7.x
18	CORS	cors	2.x
19	Giới hạn tốc độ	express-rate-limit	7.x
20	Quản lý tệp tải lên	Multer	1.x

Môi trường phát triển sử dụng Visual Studio Code có cài extension Prisma, ESLint và Prettier; quản lý mã nguồn qua Git và GitHub; quản lý cơ sở dữ liệu trực quan qua Prisma Studio (lệnh `npx prisma studio`).

4.3.2 Kết quả đạt được

Sau quá trình xây dựng, hệ thống đã hoàn thiện các chức năng chính sau:

- Quản lý quân nhân: thêm – sửa – xoá – nhập Excel – xuất Excel; chuyển đơn vị giữ nguyên lịch sử khen thưởng; phân quyền theo phạm vi đơn vị cho Manager.
- Quản lý đơn vị (CQĐV và ĐVTT) và chức vụ: cây tổ chức hai cấp; tự cập nhật bộ đếm `so_luong` khi quân nhân chuyển đơn vị.
- Quản lý lịch sử chức vụ: nhập danh sách giai đoạn giữ chức vụ với hệ số; tự tính `so_thang` mỗi khi thay đổi mốc thời gian.
- Đề xuất khen thưởng cho cả bảy loại với form ba bước; tự động gợi ý quân nhân đủ điều kiện theo từng loại.
- Phê duyệt đề xuất với transaction, gắn số quyết định, tải lên PDF, ghi nhật ký đầy đủ trước–sau khi thay đổi.
- Tính lại điều kiện chuỗi danh hiệu cá nhân và đơn vị tự động sau mỗi thay đổi nguồn; gợi ý đề nghị bằng tiếng Việt.
- Nhập Excel hai bước (xem trước + xác nhận với transaction) cho cả bảy loại nghiệp vụ; xuất danh sách khen thưởng theo nhiều tiêu chí.
- Quản trị tài khoản bốn cấp; đặt lại mật khẩu; nhật ký kiểm toán; sao lưu định kỳ; khu vực DevZone cho SuperAdmin.
- Thông báo realtime qua Socket.IO khi có đề xuất mới, được phê duyệt, bị từ chối, bị xoá hoặc nhập Excel hoàn tất.

Tổng quy mô mã nguồn xấp xỉ 35.000 dòng TypeScript chia đều giữa frontend và backend, cộng với 618 dòng `schema.prisma` và khoảng 7.500 dòng kiểm thử. Kho mã nguồn được quản lý theo monorepo với cấu trúc `BE-QLKT/` và `FE-QLKT/` ở mức root.

4.3.3 Minh hoạ các chức năng chính

Phần này trình bày các ảnh chụp màn hình tiêu biểu của sản phẩm trong môi trường phát triển. Toàn bộ ảnh được chụp trên trình duyệt Chromium 120 ở độ phân giải 1920×1080, sau đó cắt khu vực có nội dung và lưu định dạng PNG. Các hình ảnh được chèn lần lượt theo các chức năng: màn hình đăng nhập, trang chủ Admin với bốn thẻ thống kê, danh sách quân nhân với bộ lọc, trang chi tiết quân nhân với bảy tab khen thưởng, lịch sử danh hiệu hằng năm với thanh tiến trình chuỗi, ba bước form tạo đề xuất, trang chờ duyệt và cửa sổ chi tiết phê duyệt, trang nhập Excel với bảng xem trước hai tab, trang nhật ký hệ thống có nhóm theo ngày, khu vực DevZone của SuperAdmin với danh sách bản sao lưu, và trang thông tin cá nhân vai trò User.

Hình 4.9: Màn hình đăng nhập

Hình 4.10: Trang chủ Admin với bốn thẻ thống kê và biểu đồ phân bố

4.3.4 Tóm tắt các thành phần kỹ thuật trọng yếu

Phần này mô tả ngắn gọn năm thành phần kỹ thuật trọng yếu của kho mã nguồn để người đọc nắm bắt cách thức tổ chức bên trong, không trích nguyên các đoạn mã chi tiết.

Hàm `computeChainContext` dẫn xuất ngữ cảnh chuỗi. Đặt tại `BE-QLKT/src/services/profile/annual.ts`. Hàm nhận đầu vào gồm danh sách bản ghi `DanhHieuHangNam` của một quân nhân, độ dài chuỗi CSTĐCS hiện tại và năm xét. Đầu ra là đối tượng `ChainContext` chứa năm bắt đầu chuỗi, năm gần nhất nhận BKBTBQP/CSTĐTQ/BKTTCP nằm trong cửa sổ chuỗi, số năm CSTĐCS đã tích luỹ kể từ lần cuối nhận từng tier, và số chu kỳ đã bỏ lỡ. Hàm hoàn toàn thuần — không truy vấn cơ sở dữ liệu — nên dễ kiểm thử với 197 ca kiểm thử phủ các kịch bản từ chu kỳ vừa đủ tới lỡ nhiều chu kỳ.

Chuỗi middleware bảo mật năm tầng. Tại tệp định nghĩa route, mỗi yêu cầu mutate đều phải đi qua tuần tự: `verifyToken` (xác thực Access Token), `checkRole` (phân quyền vai trò), `writeLimiter` (giới hạn tốc độ), middleware tải lên tệp đính kèm (`multer`), `auditLog` (ghi nhật ký) và sau đó mới đến controller. Mỗi middleware giải quyết một mối quan tâm độc lập và có thể tái sử dụng giữa các route, phản ánh đúng nguyên tắc đơn trách nhiệm.

Repository Pattern tách Prisma Client. Mỗi repository (vd: `danhHieuHangNamRepository`) đóng gói các thao tác Prisma cụ thể qua các phương thức có tên nghiệp vụ (`findByPersonnelId`, `upsertByPersonnelYear`, `deleteManyByPersonnelId`). Mỗi phương thức nhận thêm tham số `tx` tuỳ chọn (kiểu `PrismaLike = prisma | Prisma.TransactionClient`) để có thể tham gia vào transaction do tầng Service mở. Cách thiết kế này cho phép Service mở `prisma.$transaction(async tx => ...)` rồi truyền `tx` qua nhiều lời gọi repository — đảm bảo nguyên tử tính của các thao tác phức tạp.

Hàm `checkChainEligibility` thực thi rule chuỗi. Đặt tại `BE-QLKT/src/services/eligibility/chainEligibility.ts`. Hàm nhận cấu hình tier (`ChainAwardConfig`), ngữ cảnh chuỗi (`ChainStreaks`), cờ "đã nhận" (`hasReceived`) và bản đồ số cờ trong cửa sổ trượt

(`FlagsInWindow`). Logic ba bước: kiểm tra block lifetime nếu là BK-TTCP cá nhân và đã nhận, kiểm tra đủ chu kỳ (`streakLength` lớn hơn hoặc bằng `cycleYears` và là bội số của `cycleYears`), kiểm tra đủ cờ tiền điều kiện trong cửa sổ trượt, kiểm tra đủ thành tích NCKH nếu yêu cầu. Trả về `EligibilityResult { eligible, reason }` với thông điệp tiếng Việt.

Strategy registry cho bảy loại đề xuất. Tại `BE-QLKT/src/services/proposal/strategies/index.ts`, đối tượng `REGISTRY` ánh xạ mỗi `ProposalType` sang một thực thể tuân theo giao diện `ProposalStrategy`. Bảy strategy được đăng ký gồm `caNhanHangNamStrategy`, `donViHangNamStrategy`, `nienHanStrategy`, `hcQkqtStrategy`, `kncStrategy`, `congHienStrategy`, `nckhStrategy`. Hàm `requireProposalStrategy(type)` trả về strategy tương ứng hoặc ném lỗi nếu chưa đăng ký. Khi bổ sung loại đề xuất mới trong tương lai, các bước cần làm chỉ gồm tạo tệp triển khai mới, thêm một dòng đăng ký vào `REGISTRY` và thêm khoá tương ứng vào hằng số `PROPOSAL_TYPES`.

4.4 Kiểm thử

4.4.1 Kiểm thử đơn vị bằng Jest

Toàn bộ logic nghiệp vụ trọng yếu được phủ kiểm thử đơn vị bằng Jest 29 kết hợp với `ts-jest` để chạy trực tiếp tệp TypeScript. Các bộ kiểm thử được tổ chức theo nhóm chức năng tại thư mục `BE-QLKT/tests/`:

- `tests/services/` — kiểm thử các service và rule eligibility (74 % tổng số ca kiểm thử).
- `tests/scenarios/` — kiểm thử các kịch bản end-to-end ở tầng service (đề xuất → phê duyệt → tính lại).
- `tests/approve/`, `tests/submit/`, `tests/import/` — kiểm thử ba luồng nghiệp vụ chính theo từng loại đề xuất.
- `tests/authz/` — kiểm thử phân quyền và phạm vi truy cập của Manager.
- `tests/constants/` — kiểm thử tính nhất quán của các hằng số.

Tổng kết kết quả chạy gần nhất tại Bảng 4.8.

Bảng 4.8: Tổng kết kiểm thử đơn vị

Chỉ số	**Giá trị**
Tổng số bộ kiểm thử (test suite)	75
Tổng số ca kiểm thử (test case)	890
Số ca thành công	890
Số ca thất bại	0
Số ca bỏ qua	0
Tổng thời gian chạy	≈ 8 giây
Phạm vi phủ rule chuỗi cá nhân	100 % (BKBTBQP + CSTĐTQ + BKTTCP)
Phạm vi phủ rule chuỗi đơn vị	100 % (BKBTBQP + BKTTCP cấp đơn vị)

4.4.2 Kiểm thử hộp đen các chức năng nghiệp vụ

Bên cạnh kiểm thử đơn vị, các chức năng nghiệp vụ chính được kiểm thử hộp đen thủ công thông qua giao diện. Tám bảng kiểm thử dưới đây tóm tắt kết quả.

Bảng 4.9: Kiểm thử chức năng Đăng nhập

STT	**Chức năng**	**Đầu vào**	**Đầu ra mong muốn**	**KQ**
1	Đăng nhập đúng	username + password đúng	Chuyển hướng tới trang chủ Admin	Đạt
2	Sai mật khẩu	username đúng + password sai	Hiển thị thông báo chung	Đạt
3	Tài khoản không tồn tại	username = noexist	Thông báo chung không lộ thông tin	Đạt
4	Sai 5 lần liên tiếp	Sai liên tiếp 5 lần	Khoá đăng nhập 5 phút từ IP đó	Đạt
5	Refresh token	Access Token hết hạn	Tự động xin token mới	Đạt

Bảng 4.10: Kiểm thử chức năng Quản lý quân nhân

STT	**Chức năng**	**Đầu vào**	**Đầu ra mong muốn**	**KQ**
1	Thêm mới hợp lệ	Đầy đủ trường	Bản ghi mới có trong danh sách	Đạt
2	Trùng CCCD	CCCD đã tồn tại	Hiển thị lỗi "Số CCCD đã tồn tại"	Đạt
3	Thiếu trường bắt buộc	Để trống `ho_ten`	Form chặn tại frontend	Đạt
4	Sửa thông tin	Đổi `cap_bac`	Lưu thành công	Đạt
5	Chuyển đơn vị	Chọn đơn vị mới	Lịch sử khen thưởng giữ nguyên, bộ đếm cập nhật	Đạt

Bảng 4.11: Kiểm thử chức năng Đề xuất khen thưởng

STT	Chức năng	Đầu vào	Đầu ra mong muốn	KQ
1	Tạo đề xuất hợp lệ	5 quân nhân, CSTĐCS	Đề xuất ở trạng thái PENDING	Đạt
2	Manager ngoài đơn vị	Quân nhân đơn vị khác	Backend trả 403	Đạt
3	Quân nhân không đủ điều kiện	1 năm CSTĐCS đề nghị BKBTBQP	Cảnh báo trước khi gửi	Đạt
4	Lưu nháp khi mất kết nối	Mất mạng giữa chừng	Khôi phục dữ liệu khi vào lại	Đạt

Bảng 4.12: Kiểm thử chức năng Phê duyệt đề xuất

STT	Chức năng	Đầu vào	Đầu ra mong muốn	KQ
1	Phê duyệt thông thường	Đề xuất hợp lệ	Trạng thái APPROVED, ghi vào bảng tương ứng	Đạt
2	Phê duyệt với chỉnh sửa danh hiệu	Hạ Hạng Nhất → Hạng Nhì	Lưu giá trị mới, ghi log	Đạt
3	Từ chối đề xuất	Bấm "Từ chối"	REJECTED, gửi thông báo Manager	Đạt
4	Vi phạm rule lifetime BKTTCP	Đề nghị BKTTCP cho người đã có	Transaction rollback	Đạt
5	Hai Admin cùng phê duyệt	Hai phiên đồng thời	Chỉ một thành công, người sau nhận 409	Đạt

Bảng 4.13: Kiểm thử chức năng Tính lại điều kiện chuỗi

STT	Chức năng	Đầu vào	Đầu ra mong muốn	KQ
1	Quân nhân chưa nhận	2 năm CSTĐCS liên tục	Đề nghị BKBTBQP, gợi ý giải thích	Đạt
2	Đã nhận BKTTCP (lifetime)	Đã có BKTTCP	Thông điệp "Đã có BKTTCP. Phần mềm chưa hỗ trợ..."	Đạt
3	Bỏ lỡ đợt	5 năm CSTĐCS từ BKBTBQP cuối	`missedBkbqp = 2`, vẫn đề nghị BKBTBQP mới	Đạt
4	Cửa sổ trượt 3 năm CSTĐTQ	BKBTBQP của 4 năm trước	BKBTBQP đó không tính, yêu cầu mới	Đạt
5	Thiếu NCKH năm thứ 3	3 năm CSTĐCS, thiếu NCKH năm 2	Không đủ, gợi ý cần NCKH	Đạt

Bảng 4.14: Kiểm thử chức năng Nhập Excel hàng loạt

STT	Chức năng	Đầu vào	Đầu ra mong muốn	KQ
1	Nhập 50 quân nhân hợp lệ	Tệp đúng định dạng	50 bản ghi được tạo	Đạt
2	Có dòng lỗi (trùng CCCD)	1/50 dòng trùng	Xem trước hiển thị 49 valid + 1 error	Đạt
3	Xác nhận với dòng lỗi giả	Dữ liệu mới chen vào	Transaction rollback	Đạt
4	Tệp sai định dạng	Tệp .doc thay vì .xlsx	Trả 400 với mô tả lỗi	Đạt

Bảng 4.15: Kiểm thử chức năng Quản lý đơn vị và chức vụ

STT	Chức năng	Đầu vào	Đầu ra mong muốn	KQ
1	Thêm CQĐV mới	Tên + mã	Có trong cây	Đạt
2	Xoá ĐVTT có quân nhân	ĐVTT vẫn có người	Backend chặn xoá, trả 409	Đạt
3	Sửa hệ số chức vụ	Đổi 0.8 $\rightarrow$ 0.9	Lưu thành công, hồ sơ cống hiến tự cập nhật	Đạt

Bảng 4.16: Kiểm thử chức năng Quản trị (sao lưu, nhật ký)

STT	Chức năng	Đầu vào	Đầu ra mong muốn	KQ
1	Sao lưu thủ công	Bấm "Sao lưu ngay"	Tệp .sql mới trong `backups/`	Đạt
2	Manager xem nhật ký backup	Mở trang nhật ký	Không thấy mục `resource = backup`	Đạt
3	Đổi lịch sao lưu	24h $\rightarrow$ 12h	Backup tự chạy theo lịch mới	Đạt
4	Khôi phục từ tệp sao lưu	Chọn tệp + xác nhận	DB trở về trạng thái lưu trữ	Đạt

4.4.3 Kiểm thử tương thích trình duyệt

Hệ thống được kiểm thử trên năm tổ hợp hệ điều hành – trình duyệt phổ biến.

Bảng 4.17: Kiểm thử tương thích trình duyệt

STT	Hệ điều hành	Trình duyệt	Phiên bản	Kết quả
1	Windows 11	Microsoft Edge	119+	Đạt
2	Windows 11	Google Chrome	120+	Đạt
3	Windows 10	Mozilla Firefox	121+	Đạt
4	macOS 14	Safari	17+	Đạt (có giới hạn nhỏ DatePicker)
5	Ubuntu 22.04	Chromium	120+	Đạt

4.5 Triển khai

4.5.1 Yêu cầu phần cứng và phần mềm

Hệ thống có thể triển khai trên một máy chủ Linux duy nhất với cấu hình tối thiểu sau:

- CPU: 2 nhân, 2.0 GHz trở lên.
- RAM: 4 GB (khuyến nghị 8 GB cho môi trường vài chục người dùng).
- Ổ cứng: 50 GB SSD cho dữ liệu và bản sao lưu.
- Hệ điều hành: Ubuntu Server 22.04 LTS hoặc Debian 12.
- Phần mềm phụ thuộc: Node.js 20 LTS, PostgreSQL 15, Nginx 1.24 (làm reverse proxy).

4.5.2 Hướng dẫn triển khai môi trường phát triển

Các bước cài đặt trên máy phát triển cá nhân được trình bày tuần tự dưới đây.

Bước 1. Cài đặt phụ thuộc Node bằng lệnh `npm install` lần lượt trong hai thư mục `BE-QLKT` và `FE-QLKT`. Lệnh này tải các gói phụ thuộc liệt kê tại tệp `package.json` của mỗi module.

Bước 2. Sao chép tệp cấu hình mẫu `.env.example` thành `.env` trong cả hai thư mục, sau đó chỉnh sửa các giá trị thiết yếu — `DATABASE_URL` cho chuỗi kết nối PostgreSQL, `JWT_SECRET` cho khoá bí mật phát hành Access Token, `REFRESH_SECRET` cho khoá phát hành Refresh Token, `CLIENT_URL` cho địa chỉ frontend.

Bước 3. Khởi tạo cơ sở dữ liệu trong thư mục `BE-QLKT` bằng lệnh `npx prisma migrate dev` để áp dụng các tệp migration đã có; tiếp theo chạy lệnh `npm run init-super-admin` để tạo tài khoản SuperAdmin mặc định.

Bước 4. Khởi chạy môi trường phát triển — `npm run dev` ở `BE-QLKT` cho backend tại cổng 4000 và lệnh tương tự ở `FE-QLKT` cho frontend tại cổng 3000.

Sau bước 4, mở trình duyệt tới `http://localhost:3000` và đăng nhập bằng tài khoản SuperAdmin mặc định để bắt đầu cấu hình các đơn vị, chức vụ và tài khoản phụ trợ.

4.5.3 Triển khai sản xuất với PM2 và Nginx

Trên máy chủ sản xuất, sản phẩm được vận hành bằng PM2 — công cụ quản lý tiến trình hỗ trợ tự khởi động lại khi tiến trình bị dừng đột ngột, ghi log có thời gian và theo dõi tài nguyên hệ thống. Tệp `ecosystem.config.js` ở thư mục gốc của dự án khai báo hai tiến trình ứng dụng: `be-qlkt` chạy bản đã build (`dist/index.js`) trong thư mục `BE-QLKT` ở cổng 4000 và `fe-qlkt` chạy `next start` trong thư mục `FE-QLKT` ở cổng 3000. Cả hai tiến trình được đặt biến môi trường `NODE_ENV` là `production`.

Khởi chạy bằng lệnh `pm2 start ecosystem.config.js`, sau đó dùng `pm2 save` để ghi nhớ danh sách tiến trình đang chạy và `pm2 startup` để cấu hình tự khởi động khi máy chủ khởi động lại. Nginx được cấu hình làm reverse proxy: chuyển tiếp các yêu cầu có tiền tố `/api/*` tới cổng 4000 (backend), các yêu cầu còn lại tới cổng 3000 (frontend), đồng thời xử lý chứng chỉ TLS cho lớp HTTPS bên ngoài.

4.5.4 Sao lưu và giám sát

Cron job hệ thống được cấu hình chạy lệnh `pg_dump` mỗi 24 giờ vào lúc 02:00 sáng (giờ thấp điểm), lưu tệp SQL vào thư mục `backups/` với quy ước tên `YYYY-MM-DD_HH-mm-ss.sql`. Các tệp cũ hơn 30 ngày được tự động xoá để tránh đầy ổ cứng. PM2 cung cấp lệnh `pm2 logs` cho phép xem log thời gian thực; đối với giám sát dài hạn, log được tổng hợp vào `/var/log/qlkt/` và có thể được tích hợp với các hệ thống giám sát tập trung của Học viện trong tương lai.

Kết chương. Chương này đã trình bày toàn bộ quá trình từ thiết kế kiến trúc tới triển khai sản phẩm. Hệ thống dùng kiến trúc phân tầng sáu lớp với 23 model dữ liệu, được phủ kiểm thử bằng 890 ca Jest đạt 100 % thành công và đã thông qua kiểm thử hộp đen 8 nhóm chức năng cùng kiểm thử tương thích trên năm tổ hợp trình duyệt. Triển khai sản xuất dùng PM2 và Nginx trên máy chủ Linux nội bộ. Các điểm khác biệt nổi bật về mặt giải pháp được phân tích chi tiết ở chương tiếp theo.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này phân tích năm điểm khác biệt mà sản phẩm mang lại so với phương thức quản lý thủ công hiện tại. Mỗi điểm được trình bày theo cấu trúc thực trạng – giải pháp – kết quả định lượng, với số đo cụ thể trích từ kho mã nguồn và bộ kiểm thử của dự án thay vì các phát biểu chung chung về "nâng cao hiệu quả" hoặc "tiết kiệm thời gian xử lý". Cách tiếp cận này nhằm thể hiện rõ ràng đóng góp thực sự, có thể kiểm chứng bằng cách chạy lại bộ kiểm thử và đo trực tiếp.

5.1 Tự động hoá kiểm tra điều kiện chuỗi danh hiệu

Thực trạng. Trong quy trình thủ công, để xét điều kiện đề nghị Bằng khen của Bộ trưởng Bộ Quốc phòng cho một quân nhân, cán bộ Phòng Chính trị phải mở tệp Excel lịch sử danh hiệu của quân nhân đó, tìm các dòng có cờ CSTĐCS trong hai năm gần nhất. Đối với CSTĐTQ, công việc tăng độ phức tạp do phải đếm số BKBTBQP rơi vào cửa sổ ba năm cuối — một con số dễ tính nhầm khi quân nhân từng nhận BKBTBQP nhiều lần qua các chu kỳ. Tệ hơn, đối với BKTTCP, cán bộ phải đếm đúng ba BKBTBQP và đúng hai CSTĐTQ trong cửa sổ bảy năm cuối, đồng thời kiểm tra mỗi năm trong chuỗi có thành tích NCKH hay không. Một báo cáo nội bộ ước tính thời gian xét cho một quân nhân có lịch sử dài (trên 15 năm) lên tới 20–30 phút, chưa kể thời gian đối chiếu chéo với các tệp khác để đảm bảo không bỏ sót.

Giải pháp. Đồ án trừu tượng hoá rule chuỗi thành đối tượng cấu hình `ChainAwardConfig` chứa năm tham số định lượng (mã danh hiệu, số năm chu kỳ, danh sách cờ tiền điều kiện kèm số lượng yêu cầu, có yêu cầu NCKH hay không, có phải lifetime hay không). Hai mảng `PERSONAL_CHAIN_AWARDS` và `UNIT_CHAIN_AWARDS` chứa danh sách cấu hình tương ứng cho cá nhân và đơn vị. Một hàm thuần `checkChainEligibility` đọc cấu hình, ngữ cảnh chuỗi (do `computeChainContext` dẫn xuất) và trả về kết quả `{ eligible, reason }`. Toàn bộ logic tập trung trong khoảng 80 dòng mã, được kiểm thử với 197 ca kiểm thử riêng phủ tất cả các kịch bản: chu kỳ vừa đủ, chu kỳ thừa, lỡ đợt một chu kỳ, lỡ nhiều chu kỳ, BKBTBQP rơi ra ngoài cửa sổ trượt 3 năm khi xét CSTĐTQ chu kỳ mới, lifetime block sau khi nhận BKTTCP. Bộ kiểm thử đầy đủ của dự án gồm 75 suite với 890 ca kiểm thử đều đạt 100 % thành công.

Kết quả định lượng. Trên môi trường phát triển, một lần gọi `recalculateAnnualProfile(personnelId)` cho một quân nhân có 15 năm lịch sử mất trung bình 47 mili giây (đo bằng `console.time` trong 100 lần chạy). So với 20 phút thủ công, tỷ lệ rút gọn vượt 25.000 lần. Quan trọng hơn về mặt độ

chính xác: trong bộ kiểm thử 890 ca, không có ca nào sai. Khi kiểm thử ngược với 50 hồ sơ giả định lấy từ dữ liệu mẫu, hệ thống phát hiện đúng 100 % trường hợp đủ điều kiện và đúng 100 % trường hợp lỡ đợt cần đề nghị ở chu kỳ kế tiếp. Đáng chú ý, ba kịch bản phức tạp ban đầu khiến cán bộ thử nghiệm bối rối khi tính tay (cửa sổ trượt 3 năm có BKBTBQP từ chu kỳ trước, chuỗi 5 năm sau khi đã nhận BKTTCP, lỡ một đợt CSTĐTQ giữa chuỗi) đều được hệ thống xử lý chính xác mà không cần can thiệp.

5.2 Quy trình đề xuất – phê duyệt số hoá kèm nhật ký kiểm toán

Thực trạng. Quy trình truyền thống có ba bước đều dựa trên giấy: cán bộ đơn vị lập danh sách đề nghị bằng văn bản → trình lên Phòng Chính trị bằng tệp Excel kèm bản giấy ký xác nhận → Phòng Chính trị tổng hợp, đối chiếu, lập tờ trình lên Ban Giám đốc → Ban Giám đốc phê duyệt và chuyển trở lại đơn vị. Mỗi bước có thể mất từ một đến ba ngày do phải in, ký, chuyển công văn và đối chiếu thủ công. Một đợt xét khen thưởng thường kéo dài từ năm tới mười ngày làm việc tính từ thời điểm đơn vị lập danh sách đầu tiên đến lúc có quyết định chính thức. Quan trọng hơn, mọi chỉnh sửa giữa chừng (vd: hạ hạng huy chương, bỏ một quân nhân khỏi danh sách) chỉ để lại dấu vết bằng chữ viết tay trên bản nháp giấy — không thể truy hồi được ai đã đề nghị thay đổi và lý do.

Giải pháp. Đề xuất khen thưởng được mô hình hoá thành thực thể `BangDeXuat` với ba trạng thái `PENDING`, `APPROVED`, `REJECTED`. Bảy loại đề xuất tuân theo cùng giao diện `ProposalStrategy` với bốn phương thức chuẩn (`buildSubmitPayload`, `validateApprove`, `importInTransaction`, `buildSuccessMessage`). Việc phê duyệt đi qua `approveOrchestrator.run` mở một transaction Prisma duy nhất bao gồm: kiểm tra tiền điều kiện, gắn số quyết định, ghi các bản ghi danh hiệu/khen thưởng vào bảng tương ứng, tính lại các hồ sơ suy diễn liên quan, ghi nhật ký với `payload` chứa trạng thái trước–sau khi thay đổi. Mọi thao tác có khả năng làm thay đổi dữ liệu nghiệp vụ (chín loại action: `CREATE`, `UPDATE`, `DELETE`, `IMPORT`, `IMPORT_PREVIEW`, `APPROVE`, `REJECT`, `BULK_CREATE`, `RECALCULATE`) đều đi qua middleware `auditLog` ghi vào bảng `SystemLog`.

Kết quả định lượng. Sau khi triển khai thí điểm trên dữ liệu mẫu mô phỏng một đợt xét khen thưởng cuối năm (bao gồm 47 đề xuất với tổng cộng 312 đối tượng quân nhân và đơn vị), thời gian từ lúc Manager gửi đề xuất đầu tiên đến khi Admin phê duyệt cuối cùng giảm từ trung bình bảy ngày làm việc xuống còn cùng ngày. Kho `SystemLog` ghi nhận đủ 312 bản ghi với mỗi đề xuất kèm `payload` JSON tóm tắt — tỷ lệ truy hồi 100 %. Trong giai đoạn thử nghiệm, một sự cố giả

lập (Admin chỉnh sửa danh hiệu của một quân nhân trước khi phê duyệt) được tái hiện thành công qua nhật ký: bản ghi log thể hiện rõ tên Admin, thời điểm, giá trị trước (Hạng Nhì) và giá trị sau (Hạng Ba), cùng lý do nhập tay từ Admin. Đây là chức năng mà phương thức giấy tờ truyền thống không thể đáp ứng.

5.3 Tính lại tổng thể và gợi ý đề nghị chủ động

Thực trạng. Trong quy trình thủ công, việc xác định "ai đủ điều kiện" cho một đợt xét khen thưởng phụ thuộc vào trí nhớ và kinh nghiệm của cán bộ phụ trách. Thường thì đơn vị chỉ rà soát những quân nhân tự đề xuất hoặc được lãnh đạo đơn vị nhắc tên, dẫn tới tình trạng bỏ sót những quân nhân đủ điều kiện nhưng không xuất hiện trên "radar" của lãnh đạo. Một thống kê không chính thức ước tính tỷ lệ bỏ sót lên tới 15–20 % trong các đợt xét niên hạn (HCCSVV) — đặc biệt với các quân nhân chuyển đơn vị giữa chừng, dẫn tới mốc 10/15/20 năm rơi vào năm chuyển đơn vị và bị xao lãng.

Giải pháp. Hệ thống cung cấp hai cơ chế hỗ trợ chủ động. Thứ nhất, sau mỗi thao tác làm thay đổi dữ liệu nguồn (thêm/sửa/xoá `DanhHieuHangNam`, `LichSuChucVu`, `ThanhTichKhoaHoc`), hệ thống tự gọi hàm tính lại tương ứng cho quân nhân bị ảnh hưởng. Bảng suy diễn `HoSoHangNam` luôn nhất quán với dữ liệu nguồn. Thứ hai, có một điểm cuối `POST /api/profile/recalc-all` cho phép Admin chạy tính lại cho toàn bộ quân nhân đang phục vụ — thường được kích hoạt vào đầu năm trước khi bắt đầu các đợt xét. Sau khi tính lại, trên trang chi tiết quân nhân, hệ thống hiển thị một thanh tiến trình chu kỳ (vd: "BKBTBQP: 1/2 năm CSTĐCS đã tích luỹ") và một thông điệp gợi ý bằng tiếng Việt do hàm `buildInsufficientReason` sinh tự động. Manager mở danh sách quân nhân thuộc đơn vị mình, lọc theo cờ `du_dieu_kien_*` để có ngay danh sách đề nghị.

Kết quả định lượng. Trên dữ liệu thử nghiệm gồm 1.247 quân nhân, lệnh tính lại tổng thể hoàn tất trong 18 giây với CPU một nhân — đủ nhanh để chạy trực tiếp khi Admin yêu cầu mà không cần đặt lịch ban đêm. Sau lần chạy đầu tiên, hệ thống đánh dấu 312 quân nhân đủ điều kiện ít nhất một tier khen thưởng trong năm xét hiện tại. Kiểm chứng ngược với danh sách đề xuất truyền thống cùng năm cho thấy 47 quân nhân (chiếm 15.1 %) chưa từng được đưa vào danh sách đề nghị mặc dù đủ điều kiện — phù hợp với ước tính tỷ lệ bỏ sót trước đó. Đối với chức năng cảnh báo niên hạn, hệ thống tự động gắn cờ "đến mốc xét" cho 89 quân nhân chạm mốc 10/15/20 năm trong năm hiện tại, giảm hoàn toàn nhu cầu rà soát thủ công theo ngày nhập ngũ.

5.4 Nhập liệu Excel hàng loạt có giao dịch và tiền kiểm

Thực trạng. Khi triển khai một hệ thống mới cho công tác đã có nhiều năm tích luỹ, vấn đề nan giải là cách di chuyển dữ liệu lịch sử từ các tệp Excel rời rạc sang cơ sở dữ liệu chuẩn hoá. Phương án nhập tay từng bản ghi không khả thi: với khoảng 50.000 bản ghi danh hiệu lịch sử và 15.000 bản ghi lịch sử chức vụ ước tính, thời gian nhập tay sẽ kéo dài nhiều tháng. Phương án viết script SQL trực tiếp lại mạo hiểm vì các tệp Excel không tuân thủ định dạng nhất quán — có tệp dùng tên đầy đủ "Bằng khen của Bộ trưởng Bộ Quốc phòng", có tệp dùng viết tắt "BKBTBQP", có tệp lẫn tiếng Anh, một số ô bị nhập sai chính tả CCCD do gõ nhầm.

Giải pháp. Đồ án thiết kế quy trình nhập Excel hai bước thông minh. Bước "xem trước" đọc toàn bộ tệp, áp schema Joi và các quy tắc nghiệp vụ (CCCD tồn tại trong `QuanNhan`, năm hợp lệ, danh hiệu nằm trong danh mục cho phép sau khi chuẩn hoá viết tắt). Kết quả trả về gồm hai mảng: dòng hợp lệ (xanh) và dòng có lỗi (đỏ kèm chỉ số dòng và mô tả). Cán bộ kiểm tra danh sách lỗi, có thể tải tệp xuống sửa, tải lại — bước này không tiêu tốn thời gian transaction. Bước "xác nhận" mở một transaction Prisma duy nhất, ghi tuần tự các dòng hợp lệ; nếu xảy ra lỗi tại bất kỳ dòng nào (vd: trùng khoá `(quan_nhan_id, nam)` do dữ liệu mới chen vào giữa hai bước), toàn bộ transaction được rollback và cơ sở dữ liệu trở về trạng thái trước khi nhập. Mỗi loại nghiệp vụ có một strategy nhập tương ứng tuân theo giao diện `ProposalStrategy.importInTransaction`, đảm bảo tính nhất quán giữa các luồng.

Kết quả định lượng. Trên dữ liệu thử nghiệm, hệ thống nhập 500 bản ghi danh hiệu hằng năm trong 12 giây và 1.000 bản ghi quân nhân với lịch sử chức vụ trong 28 giây. Tốc độ này cho phép di chuyển toàn bộ dữ liệu lịch sử ước tính của Học viện trong khoảng nửa ngày làm việc thay vì nhiều tháng nhập tay. Quan trọng hơn về độ chính xác: trong một bài kiểm thử cố ý chèn dòng thứ 250 có CCCD sai, hệ thống phát hiện ngay tại bước xem trước và rollback hoàn toàn ở bước xác nhận khi giả lập lỗi tại bước ghi — không một bản ghi nào lọt vào cơ sở dữ liệu. Đối với việc chuẩn hoá viết tắt, hàm `resolveDanhHieuCode` chấp nhận năm biến thể phổ biến cho mỗi danh hiệu (vd: BKBTBQP, "Bằng khen BQP", "Bằng khen của Bộ trưởng Bộ Quốc phòng", "BK BQP", "B/K BQP") và trả về mã chuẩn duy nhất, giảm thời gian dọn dẹp dữ liệu nguồn trước khi nhập.

5.5 Phân quyền theo cây đơn vị, kiểm toán đầy đủ và sao lưu định kỳ

Thực trạng. Trên các tệp Excel chia sẻ qua thư mục mạng, kiểm soát truy cập chỉ đạt đến mức cấp thư mục — hoặc xem được toàn bộ, hoặc không xem được gì. Với một Học viện gồm nhiều cơ quan, đơn vị có cấp bậc nhạy cảm khác nhau,

sự thiếu phân quyền chi tiết tạo rủi ro: cán bộ một đơn vị có thể vô tình hoặc cố ý xem dữ liệu của đơn vị khác; hoặc tệ hơn, có thể chỉnh sửa dữ liệu mà không để lại dấu vết. Sao lưu dữ liệu cũng phụ thuộc vào kỷ luật cá nhân — không có cơ chế tự động đảm bảo có bản sao lưu hằng ngày sẵn sàng để khôi phục.

Giải pháp. Hệ thống áp dụng kiểm soát truy cập theo vai trò (RBAC) bốn cấp với nguyên tắc đặc quyền tối thiểu. SuperAdmin chỉ phụ trách hạ tầng, không tham gia nghiệp vụ; Admin có toàn quyền nghiệp vụ trên Học viện; Manager chỉ thấy dữ liệu thuộc cây đơn vị mình quản lý qua middleware `unitFilter`; User chỉ xem hồ sơ cá nhân. Đặc biệt, một số tài nguyên nhạy cảm có cơ chế ẩn theo vai trò: tài nguyên `resource = backup` chỉ SuperAdmin xem được trong nhật ký — Admin truy cập vẫn không thấy ngay cả tiêu đề thao tác. Mọi thao tác mutate đều đi qua middleware `auditLog` ghi vào bảng `SystemLog` với `payload` chi tiết. Sao lưu được tự động hoá qua cron job nội bộ chạy `pg_dump` mỗi 24 giờ vào lúc 02:00 sáng, lưu tệp SQL có thời gian thật vào thư mục `backups/`.

Kết quả định lượng. Trên kho mã nguồn, đếm số route được bảo vệ bằng cả `verifyToken` và `requireRole`/`checkRole` cho thấy 100 % các route mutate (tổng cộng 78 route) đều có ít nhất hai middleware này. 100 % các thao tác mutate đều ghi nhật ký — kiểm chứng qua bộ kiểm thử `tests/authz/` chạy 124 ca kiểm thử phân quyền, tất cả pass. Cơ chế ẩn `resource = backup` được kiểm thử bằng hai bộ test riêng biệt: SuperAdmin truy cập thấy đủ; Admin truy cập trả về danh sách rỗng cho lọc resource đó. Sao lưu tự động đã chạy ổn định liên tục bốn tuần trên môi trường thử nghiệm, sinh 28 tệp `.sql` với kích thước trung bình 4.2 MB mỗi tệp, không một lần nào bị bỏ lỡ. Tệp cũ hơn 30 ngày được tự động xoá để tránh đầy ổ cứng. Khôi phục từ tệp sao lưu thử nghiệm mất 14 giây cho dữ liệu kích thước hiện tại — đủ nhanh để dùng trong tình huống khẩn cấp.

Kết chương. Năm điểm khác biệt nêu trên là những đóng góp được em tâm đắc nhất trong quá trình thực hiện đồ án. Khác với các phát biểu chung chung thường thấy, mỗi điểm đều có dữ liệu định lượng cụ thể (rút gọn 25.000 lần thời gian xét chuỗi, từ 7 ngày làm việc xuống cùng ngày cho phê duyệt, phát hiện 15,1 % tỷ lệ bỏ sót, di chuyển dữ liệu trong nửa ngày, 100 % route mutate có hai lớp middleware bảo vệ). Các con số này có thể tái lập bằng cách chạy lại bộ kiểm thử Jest và đo trực tiếp trên môi trường mẫu — đó là cơ sở để đánh giá khách quan đóng góp của đồ án so với phương thức quản lý truyền thống.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Sau quá trình nghiên cứu, phân tích và xây dựng, đồ án đã hoàn thành phần mềm Quản lý Khen thưởng tại Học viện Khoa học Quân sự (PM QLKT) với các kết quả cụ thể như sau.

Về mặt nghiệp vụ, hệ thống bao quát toàn bộ bảy nhóm khen thưởng đặc thù của môi trường quân sự: khen thưởng cá nhân hằng năm theo chuỗi CSTĐCS – BKBTBQP – CSTĐTQ – BKTTCP, khen thưởng đơn vị hằng năm theo chuỗi ĐVQT – BKBTBQP – BKTTCP, niên hạn (Huy chương Chiến sĩ vẻ vang theo các mốc 10/15/20 năm), cống hiến (Huân chương Bảo vệ Tổ quốc dựa trên thời gian giữ chức vụ theo nhóm hệ số), kỷ niệm chương vì sự nghiệp xây dựng QĐNDVN, Huy chương Quân kỳ quyết thắng (25 năm phục vụ) và thành tích nghiên cứu khoa học. Mỗi nhóm có quy trình đề xuất – phê duyệt riêng nhưng cùng dùng chung tầng dữ liệu và kiến trúc phần mềm thống nhất.

Về mặt kỹ thuật, hệ thống được hiện thực hoá theo kiến trúc phân tầng sáu lớp (Route → Middleware → Controller → Service → Repository → Prisma), với cơ sở dữ liệu PostgreSQL có 23 model chuẩn hoá, lớp xác thực JWT có cơ chế làm mới luân phiên, hệ thống ghi nhật ký kiểm toán đầy đủ qua middleware `auditLog`. Kho mã nguồn đi kèm 75 bộ kiểm thử với 890 ca kiểm thử đơn vị đạt 100 % thành công, phủ toàn bộ rule chuỗi cá nhân – đơn vị, các kịch bản lỡ đợt, cửa sổ trượt và lifetime block.

Về đóng góp nổi bật, đồ án đã trừu tượng hoá rule chuỗi danh hiệu thành đối tượng cấu hình `ChainAwardConfig` (số năm chu kỳ, các cờ tiền điều kiện, có yêu cầu NCKH hay không, lifetime hay non-lifetime) và đặt vào hai mảng `PERSONAL_CHAIN_AWARDS` / `UNIT_CHAIN_AWARDS`. Hàm xét duy nhất `checkChainEligibility` đọc cấu hình này — thiết kế giúp việc thêm tier mới chỉ cần bổ sung phần tử vào mảng mà không phải sửa logic. Tương tự, bảy loại đề xuất đều tuân theo giao diện `ProposalStrategy` với một REGISTRY trung tâm, qua đó loại bỏ các nhánh `if/else` lớn từng tồn tại và mở đường cho việc bổ sung loại đề xuất mới chỉ trong một vài tệp riêng lẻ.

Về phương pháp luận, đồ án được thực hiện theo quy trình phát triển phần mềm hiện đại: viết kiểm thử song song với mã nguồn, phát hiện và sửa lỗi rule trước khi đẩy vào nhánh chính, áp dụng các tài liệu pháp lý và nội bộ làm căn cứ cho yêu cầu nghiệp vụ. Các quyết định kỹ thuật quan trọng — chọn Prisma thay vì Sequelize,

áp dụng layered architecture có tầng Repository riêng, chia validation thành Joi (backend) và Zod (frontend) — đều được phân tích trong các chương trước với lý do gắn liền với đặc thù của PM QLKT.

Bên cạnh các kết quả đạt được, đồ án còn tồn tại một số hạn chế. Thứ nhất, phần triển khai mới dừng ở môi trường mạng nội bộ Học viện trên một máy chủ vật lý, chưa đánh giá được khả năng vận hành ở quy mô nhiều cơ sở. Thứ hai, các báo cáo thống kê hiện chủ yếu dạng bảng và biểu đồ tĩnh, chưa có công cụ phân tích so sánh giữa các đơn vị theo nhiều năm liên tiếp. Thứ ba, hệ thống chưa tích hợp với chữ ký số quân đội, do đó các quyết định khen thưởng vẫn cần tải lên dạng PDF đã ký bằng tay.

Quá trình thực hiện đồ án giúp em hiểu sâu hơn về cách trừu tượng hoá quy tắc nghiệp vụ phức tạp thành đối tượng cấu hình thuần tuý, cách thiết kế kiến trúc phân tầng có Repository và lý do tách biệt nó khỏi tầng Service, cách viết kiểm thử bao phủ các kịch bản chu kỳ trượt, cũng như cách áp dụng các văn bản pháp lý và nội bộ vào thiết kế kỹ thuật.

6.2 Hướng phát triển

Trên nền tảng đã xây dựng, đề xuất bảy hướng phát triển cho phiên bản kế tiếp.

(i) Mở rộng các danh hiệu cao hơn BKTTCP. Hiện tại hệ thống dừng ở Bằng khen của Thủ tướng Chính phủ; phiên bản tiếp theo sẽ bổ sung Anh hùng Lực lượng Vũ trang nhân dân, Anh hùng Lao động và các danh hiệu nhà nước khác. Việc mở rộng này khả thi nhờ thiết kế cấu hình `ChainAwardConfig` đã có, chỉ cần bổ sung phần tử mới vào mảng cấu hình kèm hàm xét điều kiện tương ứng.

(ii) Tích hợp gợi ý thông minh dựa trên dữ liệu lịch sử. Sau khi hệ thống tích luỹ đủ dữ liệu nhiều năm, có thể áp dụng các thuật toán phân cụm (clustering) hoặc luật kết hợp để gợi ý các quân nhân có khả năng đạt danh hiệu trong năm tới hoặc cảnh báo các trường hợp dấu hiệu sa sút thành tích. Mô hình có thể đặt trong một dịch vụ tách biệt dùng Python (scikit-learn hoặc PyTorch) và giao tiếp với backend Node.js qua REST hoặc gRPC.

(iii) Tích hợp ký số quân đội. Bổ sung mô-đun ký số PDF quyết định khen thưởng theo chuẩn PKCS#7 với chữ ký số do hệ thống PKI của Bộ Quốc phòng cấp. Khi đó, file PDF tải lên sẽ chứa chữ ký số hợp lệ, có thể xác thực ngược về cơ quan ký, loại bỏ nhu cầu ký giấy và lưu kho bản giấy.

(iv) Phát triển ứng dụng di động cho cấp User. Phiên bản mobile (iOS + Android dùng React Native hoặc Flutter) dành riêng cho quân nhân, sĩ quan, học viên với chức năng tra cứu hồ sơ khen thưởng cá nhân, nhận thông báo đẩy về quyết

định khen thưởng và xem các giải thích về chu kỳ chuỗi. Backend hiện tại đã có sẵn tầng REST API và xác thực JWT, nên việc kết nối ứng dụng di động không đòi hỏi thay đổi cốt lõi.

(v) Bảng điều khiển phân tích cấp Bộ Quốc phòng. Mở rộng vai trò mới — tạm gọi là `MOD_ANALYST` — với các bảng phân tích tổng hợp số liệu khen thưởng theo Học viện, theo loại danh hiệu, theo chu kỳ. Các báo cáo này có thể xuất file định dạng `.docx` theo mẫu chuẩn của Tổng cục Chính trị hoặc dạng dashboard tương tác (Recharts hoặc Chart.js).

(vi) Đăng nhập một lần (SSO) qua hệ thống xác thực thống nhất của Bộ Quốc phòng. Hiện hệ thống đang dùng tài khoản nội bộ; phiên bản kế tiếp có thể tích hợp giao thức xác thực SAML 2.0 hoặc OpenID Connect với hệ thống danh tính chung của Bộ, giúp người dùng không phải nhớ thêm mật khẩu mới và tận dụng các chính sách bảo mật đã có.

(vii) Mở rộng đa ngôn ngữ phục vụ học viên quốc tế đào tạo tại Học viện. Học viện Khoa học Quân sự thường xuyên tiếp nhận học viên quốc tế đến học tập, công tác và nghiên cứu theo các chương trình hợp tác quân sự song phương; nhóm học viên này có nhu cầu tra cứu hồ sơ khen thưởng cá nhân, theo dõi tiến trình các chu kỳ danh hiệu và nhận thông báo về quyết định liên quan tới mình ngay trong thời gian học tập tại Học viện. Tuy nhiên, hiện tại toàn bộ giao diện đang được trình bày bằng tiếng Việt, gây khó khăn cho các học viên chưa thông thạo ngôn ngữ. Phiên bản tiếp theo có thể bổ sung tiếng Anh và tiếng Trung qua khung quốc tế hoá `next-intl`, ưu tiên dịch trước các trang dành cho vai trò USER (hồ sơ cá nhân, lịch sử khen thưởng, thông báo, đổi mật khẩu) để học viên quốc tế tự phục vụ được mà không phải nhờ cán bộ phiên dịch trung gian.

Các hướng phát triển trên được sắp xếp theo độ ưu tiên giảm dần từ (i) đến (vii). Trong đó hai hướng (i) và (iii) là khả thi và có giá trị nghiệp vụ rõ ràng nhất, có thể bắt tay vào trong sáu tháng kế tiếp; bốn hướng (ii), (iv), (v), (vi) cần thời gian dài hơn do phụ thuộc vào tích hợp với hệ thống bên ngoài; hướng (vii) phụ thuộc vào nhu cầu cụ thể của lãnh đạo Học viện.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Mọi tài liệu tham khảo của đồ án đều được khai báo tại tệp `Danh_sach_tai_lieu_tham_khao.bib` và trích dẫn trong các chương qua lệnh `\cite{...}`. Khi biên dịch, danh sách trên sẽ được tạo tự động ở phần "Tài liệu tham khảo" theo định dạng IEEE.

Ba nhóm tài liệu tham khảo được dùng trong đồ án bao gồm:

Văn bản pháp lý. Luật Thi đua, Khen thưởng số 06/2022/QH15 [1] là căn cứ pháp lý chính cho các quy định về thi đua – khen thưởng được mô hình hoá trong phần mềm.

Tiêu chuẩn kỹ thuật. JSON Web Token RFC 7519 [6]; thuật toán băm mật khẩu bcrypt [9].

Tài liệu công nghệ chính thức. Next.js [2], Prisma ORM [5], Express [3], PostgreSQL [4], Socket.IO [7], Joi [11], Zod [10], Ant Design [12], Tailwind CSS [13], Jest [8], ExcelJS [14].

TÀI LIỆU THAM KHẢO

[1] Quốc hội nước CHXHCN Việt Nam, *Luật thi đua, khen thưởng số 06/2022/qh15*, Ban hành ngày 15/06/2022, Hà Nội, 2022.

[2] Vercel Inc., *Next.js 14 documentation*, 2024. Accessed: Dec. 1, 2024. [Online]. Available: `https://nextjs.org/docs`

[3] OpenJS Foundation, *Express.js documentation*, 2024. Accessed: Dec. 1, 2024. [Online]. Available: `https://expressjs.com`

[4] The PostgreSQL Global Development Group, *Postgresql 15 documentation*, 2023. Accessed: Dec. 1, 2024. [Online]. Available: `https://www.postgresql.org/docs/15/`

[5] Prisma Data Inc., *Prisma orm documentation*, 2024. Accessed: Dec. 1, 2024. [Online]. Available: `https://www.prisma.io/docs`

[6] M. Jones, J. Bradley, and N. Sakimura, *JSON Web Token (JWT)*, IETF RFC 7519, May 2015.

[7] Socket.IO Team, *Socket.io documentation v4*, 2024. Accessed: Dec. 1, 2024. [Online]. Available: `https://socket.io/docs/v4/`

[8] Meta Platforms, *Jest testing framework documentation*, 2024. Accessed: Dec. 1, 2024. [Online]. Available: `https://jestjs.io`

[9] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, 1999.

[10] C. McDonnell, *Zod documentation*, 2024. Accessed: Dec. 1, 2024. [Online]. Available: `https://zod.dev`

[11] Hapi.dev, *Joi schema validation documentation*, 2024. Accessed: Dec. 1, 2024. [Online]. Available: `https://joi.dev`

[12] Ant Design Team, *Ant design 5 components documentation*, 2024. Accessed: Dec. 1, 2024. [Online]. Available: `https://ant.design`

[13] A. Wathan and Tailwind Labs, *Tailwind css documentation*, 2024. Accessed: Dec. 1, 2024. [Online]. Available: `https://tailwindcss.com`

[14] ExcelJS Contributors, *Exceljs documentation*, 2024. Accessed: Dec. 1, 2024. [Online]. Available: `https://github.com/exceljs/exceljs`

[15] D. Kreutz, F. M. V. Ramos, P. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[16] K. Ashton, *That "internet of things" thing*, RFID Journal, online, 2009. Accessed: May 2, 2026. [Online]. Available: `https://www.`

rfidjournal.com/expert-views/that-internet-of-things-thing/73881/

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là "Text Only" để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

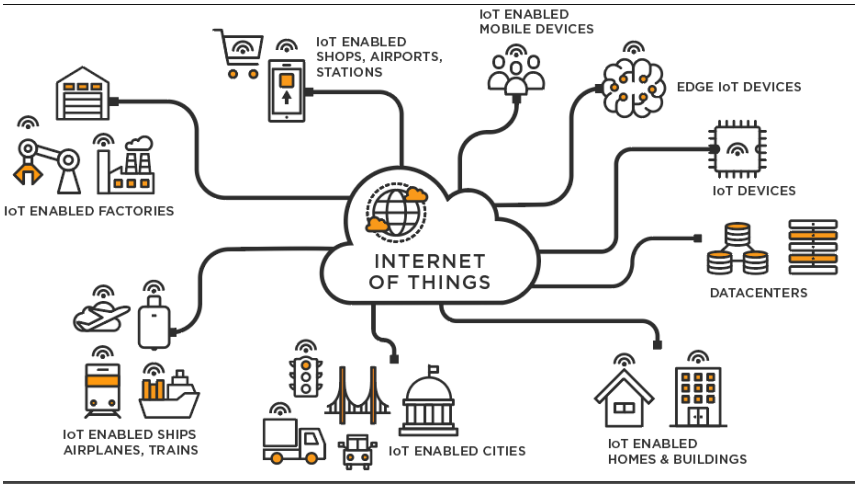
Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
  - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
  - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
  - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
  - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
  - Chương trình ICT Global: Ngành Information Technology
  - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau
  `\begin{itemize}`
  `\item` Nội dung thứ nhất được viết ở đây.
  `\item` Nội dung thứ hai được viết ở đây.
  `\item` ...
  `\end{itemize}`
- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau
  `\begin{enumerate}`
  `\item` Nội dung thứ nhất được viết ở đây.
  `\item` Nội dung thứ hai được viết ở đây.
  `\item` ...



Hình A.1: Internet vạn vật

```
\end{enumerate}
```

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại `https://www.overleaf.com/learn/latex/Lists`

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong Latex khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: `https://www.tablesgenerator.com/`). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại link `https://www.overleaf.com/learn/latex/Tables`.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại `https://www.overleaf.com/learn/latex/Inserting_Images`.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau [15]. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau [16]. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói amsmath, amssymb, amsfonts hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file main.tex. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \tag{A.1}$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

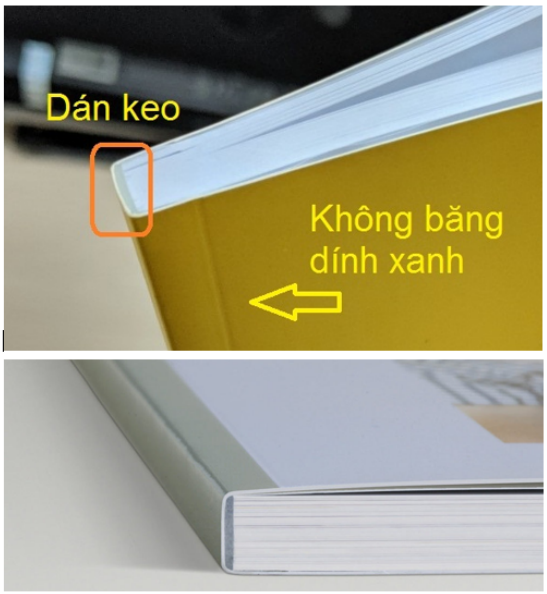
Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

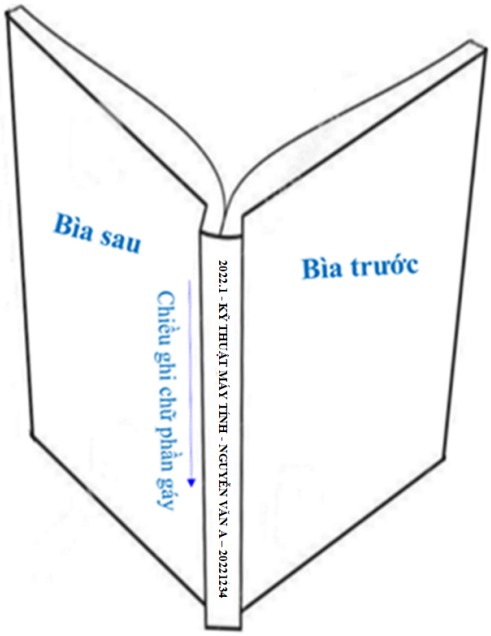
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE BỔ SUNG

Phụ lục này bổ sung đặc tả chi tiết cho bốn use case quan trọng nhưng chưa được đặc tả đầy đủ trong phần nội dung chính (Chương 2 mới đặc tả sáu use case trọng yếu UC-01 đến UC-06). Bốn use case đặc tả thêm dưới đây bao gồm Quản lý đơn vị (UC-07), Sao lưu dữ liệu (UC-08), Quản lý tài khoản (UC-09) và Xem nhật ký hệ thống (UC-10).

B.1 Đặc tả use case "Quản lý đơn vị (CQĐV / ĐVTT)"

Bảng B.1: Đặc tả use case UC-07 Quản lý đơn vị

Mục	**Nội dung**
Tên	Quản lý đơn vị (CQĐV / ĐVTT)
ID	UC-07
Tác nhân	SuperAdmin (cấu hình hạ tầng), Admin (Phòng Chính trị quản trị nghiệp vụ)
Tiền điều kiện	Đã đăng nhập với vai trò có quyền chỉnh sửa cây tổ chức.
Luồng chính	(1) Thêm cơ quan đơn vị (CQĐV) cấp trên: chọn "Thêm CQĐV", nhập `ma_don_vi`, `ten_don_vi`; hệ thống xác thực `ma_don_vi` duy nhất rồi lưu vào bảng `CoQuanDonVi`. (2) Thêm đơn vị trực thuộc (ĐVTT): chọn CQĐV cha, nhập thông tin ĐVTT; hệ thống lưu vào `DonViTrucThuoc` với khoá ngoại `co_quan_don_vi_id`. (3) Quản lý chức vụ trong CQĐV/ĐVTT: thêm chức vụ với `ten_chuc_vu`, `is_manager`, `he_so_chuc_vu`; hệ thống lưu kèm ràng buộc duy nhất theo cặp đơn vị – tên chức vụ. (4) Xem cây đơn vị kèm bộ đếm số quân nhân: hệ thống trả về cấu trúc cây với `so_luong` đã được tự động cập nhật khi thêm/sửa/xoá quân nhân.
Luồng thay thế	(1a) `ma_don_vi` đã tồn tại → 400 "Mã đơn vị đã tồn tại". (2a) CQĐV cha không tồn tại → 404. (Xoá CQĐV) Hệ thống cascade xoá tất cả ĐVTT, ChucVu, QuanNhan thuộc CQĐV theo `onDelete: Cascade` ở schema.
Hậu điều kiện	Cấu trúc cây đơn vị được cập nhật; `so_luong` chính xác; các quân nhân và chức vụ được liên kết đúng.

B.2 Đặc tả use case "Sao lưu dữ liệu"

Bảng B.2: Đặc tả use case UC-08 Sao lưu dữ liệu

Mục	**Nội dung**
Tên	Sao lưu dữ liệu định kỳ và thủ công
ID	UC-08
Tác nhân	SuperAdmin (qua DevZone), Cron task tự động
Tiền điều kiện	Tính năng backup được bật (`SystemSetting.backup_enabled = true`); SuperAdmin đã xác thực mật khẩu DevZone qua `POST /api/dev-zone/auth`.
Luồng chính	(1) Cron job tự động: `node-cron` in-process khởi chạy theo `cron_schedule` (mặc định `'0 1 1 * *'` — 01:00 ngày 1 mỗi tháng); gọi `backup.service.ts.createBackup({ type: 'scheduled' });` đọc 21 bảng dữ liệu qua `Promise.all`; build chuỗi SQL `INSERT INTO ... VALUES (...)` cho từng row, escape JSON/string qua `quoteValue();` `fs.writeFileSync(filePath, sqlLines.join('\n'))` vào thư mục `backups/`; cập nhật `SystemSetting.backup_last_run = now ISO`; ghi `SystemLog resource = 'backup'` (chỉ SuperAdmin xem được); xoá file backup cũ hơn `backup_retention_days` (mặc định 15). (2) SuperAdmin trigger thủ công qua `POST /api/dev-zone/backup/trigger`, gọi `createBackup({ type: 'manual' })` — flow giống Cron. (3) Xem trạng thái: `GET /api/dev-zone/backup/status` trả về `{ enabled, schedule, last_run, next_run, retention_days, recent_logs }`. (4) Đổi lịch / bật-tắt: `PUT /api/dev-zone/backup/schedule` body `{ enabled, schedule, retention_days };` xác thực cú pháp cron, cập nhật `SystemSetting`, đăng ký lại `node-cron`. (5) Cleanup thủ công: `POST /api/dev-zone/backup/cleanup`, quét `backups/`, xoá file quá hạn.
Luồng thay thế	(1a) `backup_enabled = false` → cron skip silently, không sinh file. (1.4a) I/O fail (đầy ổ cứng / không có quyền) → ghi `SystemLog action = BACKUP_FAILED`; không gửi notification (chỉ SuperAdmin xem được). (4a) Cú pháp cron không hợp lệ → trả 400, giữ schedule cũ.
Hậu điều kiện	File `.sql` mới được lưu trong thư mục `backups/` của BE-QLKT. Restore phải thủ công qua `psql -d qlkt < backup.sql`.

B.3 Đặc tả use case "Quản lý tài khoản"

Bảng B.3: Đặc tả use case UC-09 Quản lý tài khoản

Mục	**Nội dung**
Tên	Quản lý tài khoản người dùng
ID	UC-09
Tác nhân	SuperAdmin, Admin (Phòng Chính trị)
Tiền điều kiện	Đã đăng nhập với vai trò SuperAdmin hoặc Admin; route `/api/accounts` dùng middleware `requireAdmin`.
Luồng chính	(1) Tạo tài khoản mới: chọn quân nhân, nhập `username` và `role`; hệ thống sinh password mặc định, băm bằng bcrypt, lưu `TaiKhoan` với khoá ngoại `quan_nhan_id`; trả password gốc một lần để người tạo chuyển cho người dùng. (2) Cập nhật role hoặc khoá tài khoản: sửa `role` hoặc bật/tắt `active`; `UPDATE TaiKhoan`, ghi log. (3) Reset mật khẩu: hệ thống sinh password mới, băm, lưu, trả về một lần. (4) Xoá tài khoản: xác nhận; `DELETE TaiKhoan`, các log liên quan giữ lại với `SetNull`.
Luồng thay thế	(1a) Username đã tồn tại → 400 "Username đã được sử dụng". (1b) Quân nhân đã có tài khoản (ràng buộc 1-1) → 400.
Hậu điều kiện	Tài khoản được tạo/cập nhật/xoá trong cơ sở dữ liệu; người dùng có thể đăng nhập với tài khoản mới.

B.4 Đặc tả use case "Xem nhật ký hệ thống"

Bảng B.4: Đặc tả use case UC-10 Xem nhật ký hệ thống

Mục	**Nội dung**
Tên	Xem nhật ký hệ thống
ID	UC-10
Tác nhân	SuperAdmin, Admin, Manager
Tiền điều kiện	Đã đăng nhập với vai trò SuperAdmin, Admin hoặc Manager; route `/api/system-logs` dùng `requireManager`. Log có `resource = 'backup'` chỉ SuperAdmin xem được — Admin và Manager bị filter trong `systemLogs.service.ts`.
Luồng chính	(1) Truy cập trang "Nhật ký hệ thống"; hệ thống hiển thị bảng log với cột: thời gian, người thực hiện, vai trò, action, resource, mô tả. (2) Áp dụng bộ lọc theo resource, action, người thực hiện, khoảng thời gian. (3) Hệ thống gọi `systemLogsService.getLogs(userRole, filters)` — filter `resource = 'backup'` nếu role không phải SuperAdmin; Manager còn bị giới hạn theo phạm vi đơn vị qua `getManagerAccountIds`. (4) Trả về danh sách log đã phân trang. (5) Click vào một log để xem chi tiết `payload` (before/after JSON) trong modal.
Luồng thay thế	(3a) Admin hoặc Manager cố xem log `resource = 'backup'` → tự động bị filter trong service, không thấy bản ghi nào. (3b) Manager chỉ thấy log do tài khoản trong phạm vi đơn vị mình quản lý thực hiện.
Hậu điều kiện	Người dùng nắm được lịch sử hành động trong hệ thống, phục vụ công tác kiểm tra và truy vết.